

72.75: Aprendizaje Automático (Machine Learning)

Horario: Miércoles 16-19h- virtual sincrónico

Profesorado de la asignatura:

Francisco Sammartino, Ing. | frperez@itba.edu.ar

Carlos Bibián Noguerras, PhD | cbibian@itba.edu.ar

Plan de la sesión de hoy

- Árboles de decisión
- Hiperparámetros
- Random Forests

... en la sesión anterior

- GDA
 - Naive-Bayes
 - LDA/GDA
-

Clasificadores basados en Árboles

Árboles de Decisión y Random Forest

Carlos Bibián Nogueras, PhD
cbibian@itba.edu.ar |

Instituto Tecnológico de Buenos Aires

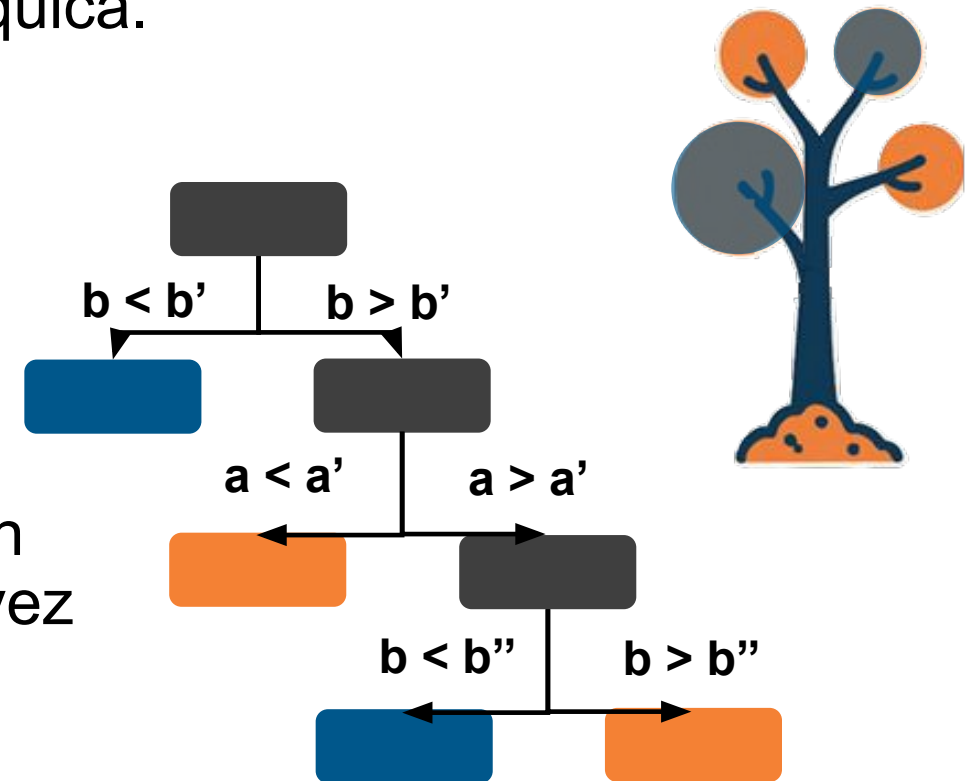
Idea principal

- Llegar a una decisión a partir de una **sucesión de preguntas de respuesta binaria**

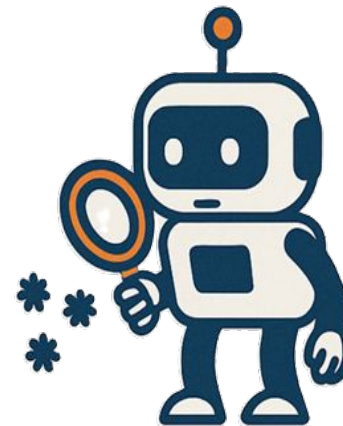
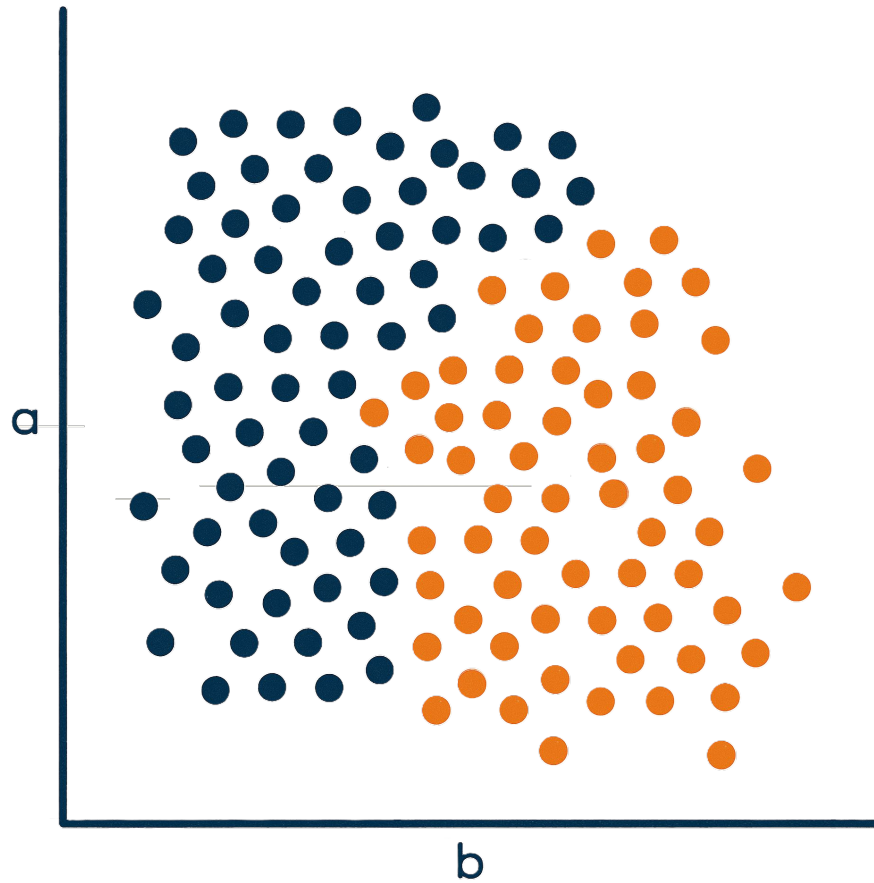


¿Qué es un Árbol de Decisión?

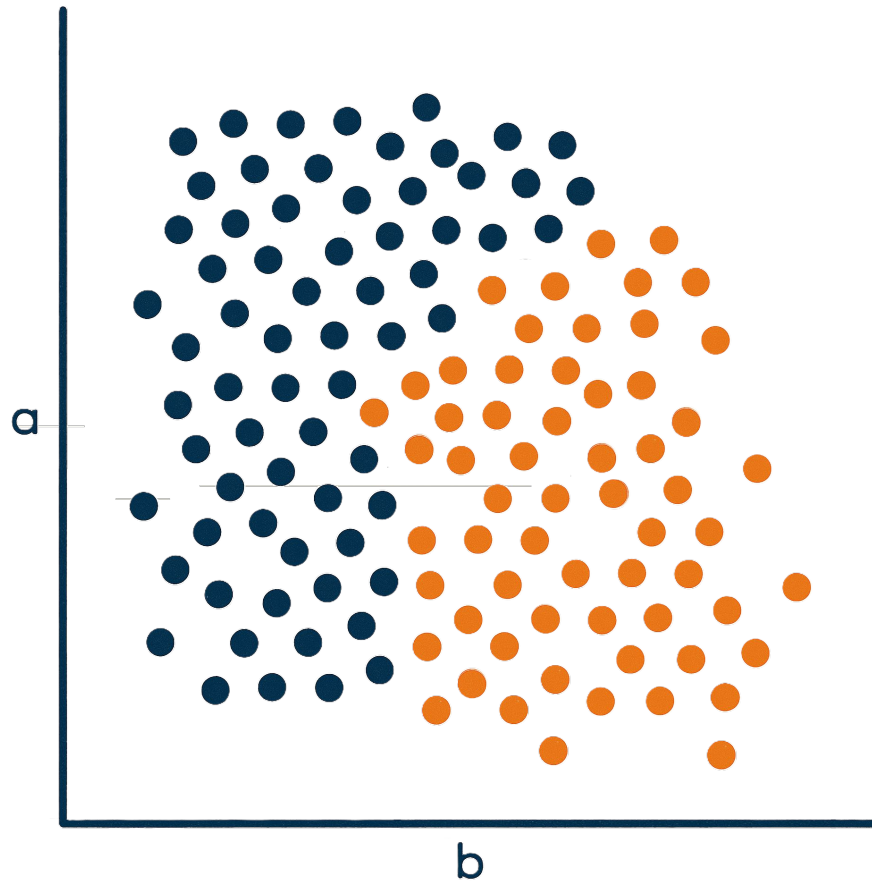
- Un modelo predictivo que toma decisiones dividiendo el espacio de los datos en reglas condicionales simples, de forma jerárquica.
- Cada decisión lleva a una rama, hasta llegar a una predicción final en una hoja.
- Secuencia de decisiones jerárquicas que dividen el espacio de características en regiones cada vez más homogéneas



¿Cómo decidimos donde colocar cada corte?



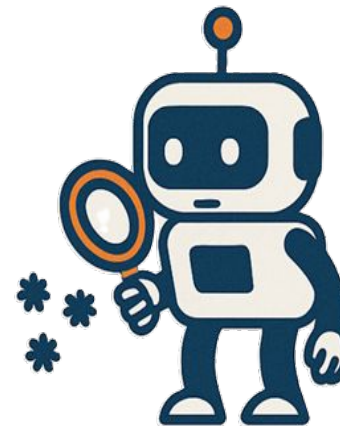
¿Cómo decidimos donde colocar cada corte?



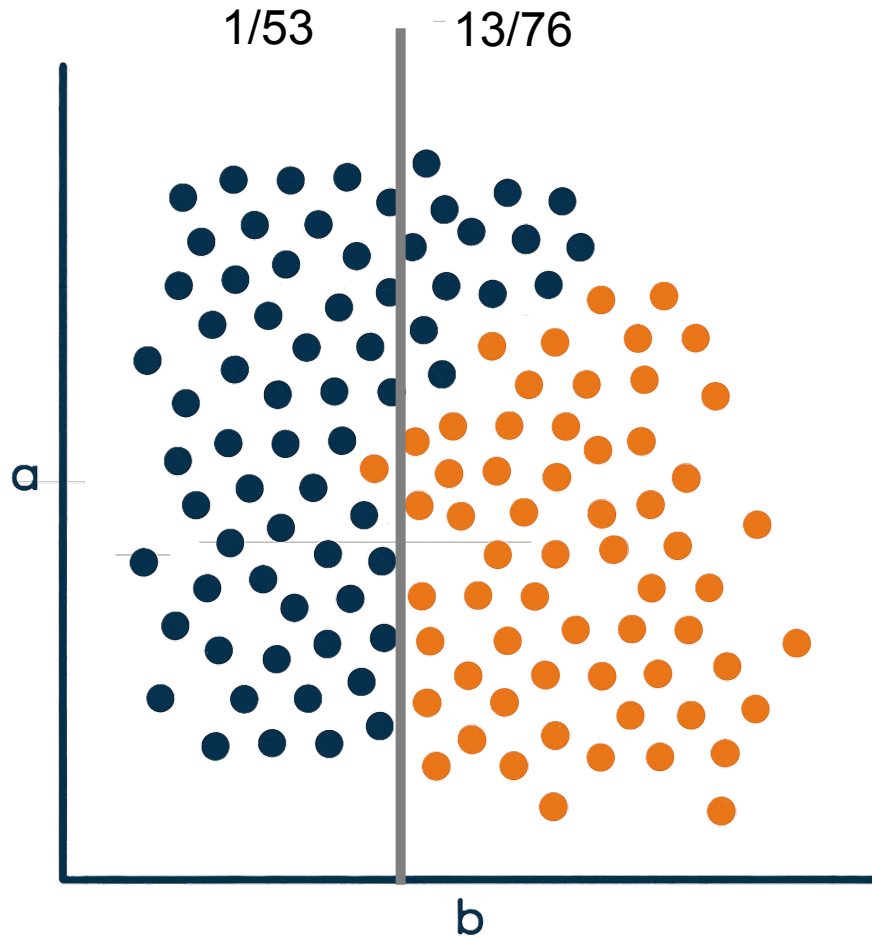
1. Calcular el error/impureza de cada posible corte (para cada característica)
Ej: calculando el porcentaje de elementos que **no** pertenecen a la clase mayoritaria en ese grupo a cada lado del corte

$$\text{Impureza del split} = \frac{n_{\text{izq}}}{n} \cdot \text{Impureza}_{\text{izq}} + \frac{n_{\text{der}}}{n} \cdot \text{Impureza}_{\text{der}}$$

2. Elegir el corte con menor error/impureza



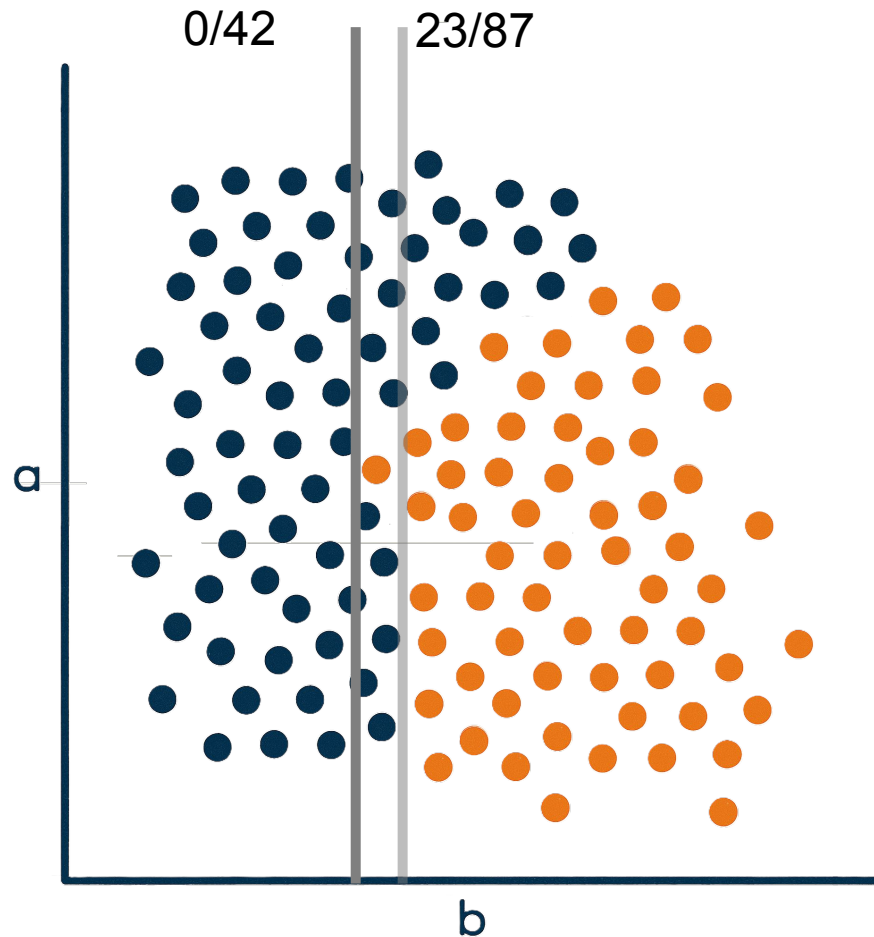
¿Cómo decidimos donde colocar cada corte?



$$Imp_{izq} = \frac{1}{53} = 0.02 \quad Imp_{dcha} = \frac{13}{76} = 0.17$$

$$Impureza\ total = \frac{53}{129} \cdot 0.02 + \frac{76}{129} \cdot 0.17 = 0.11$$

¿Cómo decidimos donde colocar cada corte?



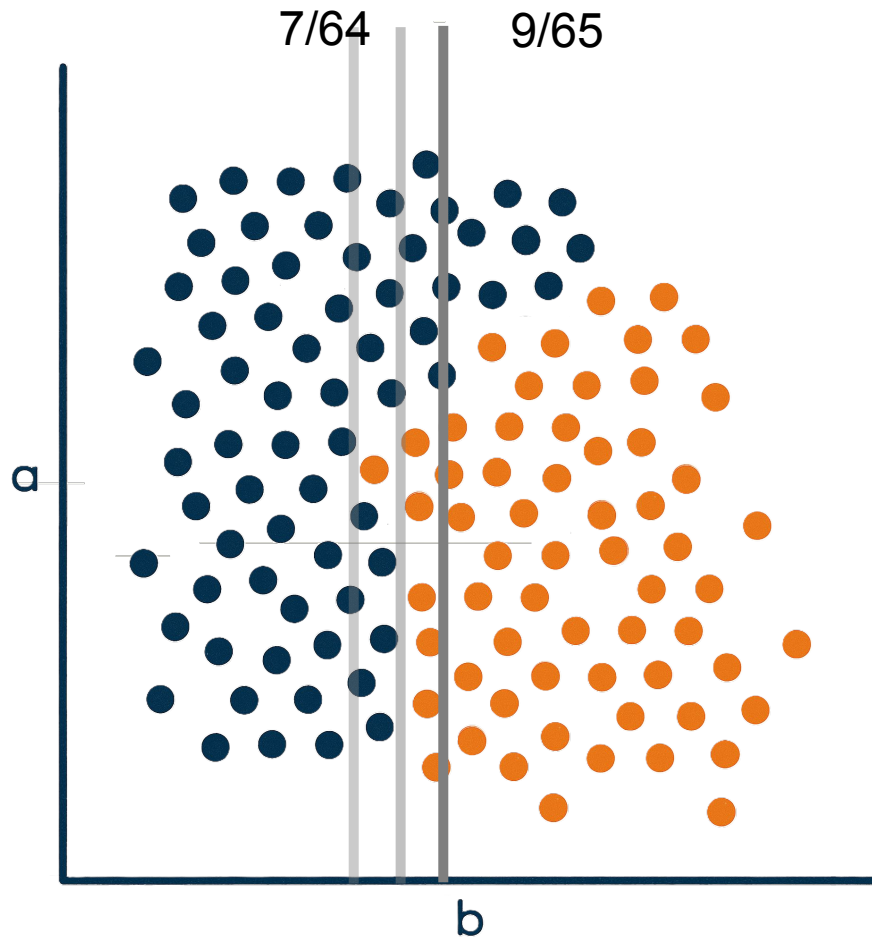
$$Imp_{izq} = \frac{1}{53} = 0.02 \quad Imp_{dcha} = \frac{13}{76} = 0.17$$

$$Impureza\ total = \frac{53}{129} \cdot 0.02 + \frac{76}{129} \cdot 0.17 = 0.11$$

$$Imp_{izq} = \frac{0}{42} = 0 \quad Imp_{dcha} = \frac{23}{87} = 0.26$$

$$Impureza\ total = \frac{42}{129} \cdot 0 + \frac{87}{129} \cdot 0.25 = 0.17$$

¿Cómo decidimos donde colocar cada corte?



$$Imp_{izq} = \frac{1}{53} = 0.02 \quad Imp_{dcha} = \frac{13}{76} = 0.17$$

$$Impureza\ total = \frac{53}{129} \cdot 0.02 + \frac{76}{129} \cdot 0.17 = 0.11$$

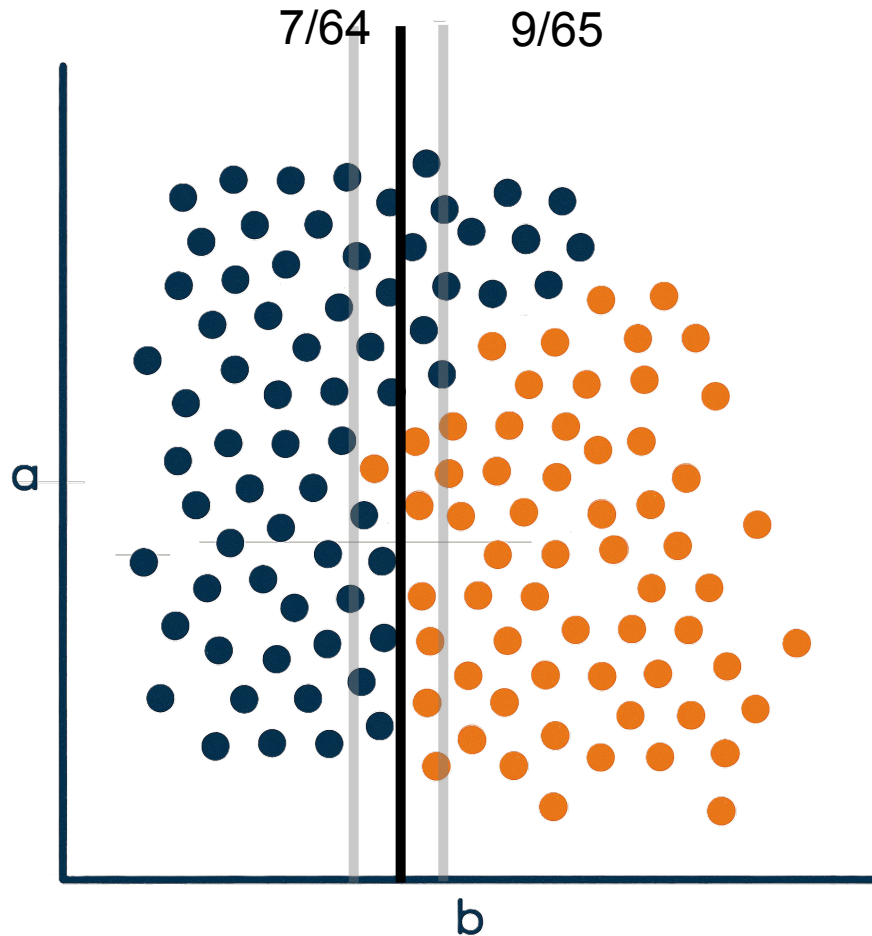
$$Imp_{izq} = \frac{0}{42} = 0 \quad Imp_{dcha} = \frac{23}{87} = 0.26$$

$$Impureza\ total = \frac{42}{129} \cdot 0 + \frac{87}{129} \cdot 0.25 = 0.17$$

$$Imp_{izq} = \frac{7}{64} = 0.11 \quad Imp_{dcha} = \frac{9}{65} = 0.14$$

$$Impureza\ total = \frac{64}{129} \cdot 0.11 + \frac{65}{129} \cdot 0.14 = 0.13$$

¿Cómo decidimos donde colocar cada corte?



$$Imp_{izq} = \frac{1}{53} = 0.02 \quad Imp_{dcha} = \frac{13}{76} = 0.17$$

$$Impureza\ total = \frac{53}{129} \cdot 0.02 + \frac{76}{129} \cdot 0.17 = 0.11$$

$$Imp_{izq} = \frac{0}{42} = 0$$

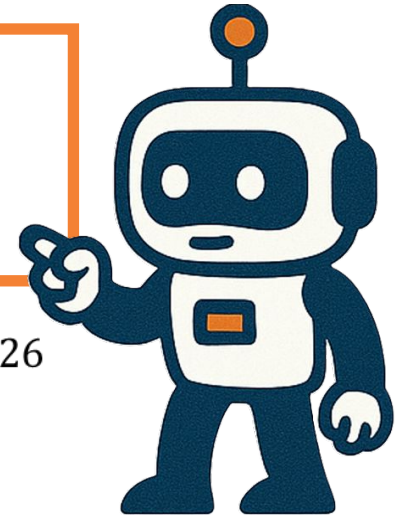
$$Imp_{dcha} = \frac{23}{87} = 0.26$$

$$Impureza\ total = \frac{42}{129} \cdot 0 + \frac{87}{129} \cdot 0.25 = 0.17$$

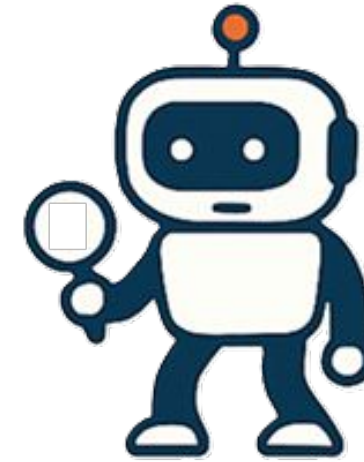
$$Imp_{izq} = \frac{7}{64} = 0.11$$

$$Imp_{dcha} = \frac{9}{65} = 0.14$$

$$Impureza\ total = \frac{64}{129} \cdot 0.11 + \frac{65}{129} \cdot 0.14 = 0.13$$



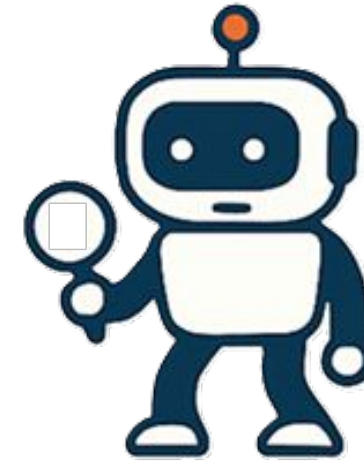
Problemas de este método



$$\text{Impureza del split} = \frac{n_{\text{izq}}}{n} \cdot \text{Impureza}_{\text{izq}} + \frac{n_{\text{der}}}{n} \cdot \text{Impureza}_{\text{der}}$$

- Insensible a los cambios dentro de nodos impuros.
 - Por ejemplo, **65/35** y **55/45** dan errores de clasificación muy parecidos (**0.35 vs 0.45**), pero el 55/45 es muchísimo peor.
- Puede que muchos cortes tengan el mismo “error de clasificación”, aunque sus distribuciones internas sean muy distintas.
- En general, árboles menos estables y precisos -> **no captura bien cuán “mezcladas” están realmente las clases.**

¿Alternativas?



$$\text{Impureza del split} = \frac{n_{\text{izq}}}{n} \cdot \text{Impureza}_{\text{izq}} + \frac{n_{\text{der}}}{n} \cdot \text{Impureza}_{\text{der}}$$

Impureza de Gini

$$Gini = \sum_{i=1}^n p_i^2$$

Entropía (Shannon)

$$entropia = - \sum_{i=1}^n p_i \log_2(p_i)$$

¿Qué es la impureza Gini?

$$Gini = \sum_{i=1}^n p_i^2$$

Donde p_i es la proporción de la clase i en el nodo y n la cantidad de clases.

- Viene de estadística económica (Corrado Gini, 1912) como medida de desigualdad:
 - Mide la desigualdad en una distribución (por ejemplo, la distribución del ingreso en una población)
- Penalizan más intensamente cuando hay mezcla de clases, incluso si una clase es claramente dominante

¿Qué es la entropía?

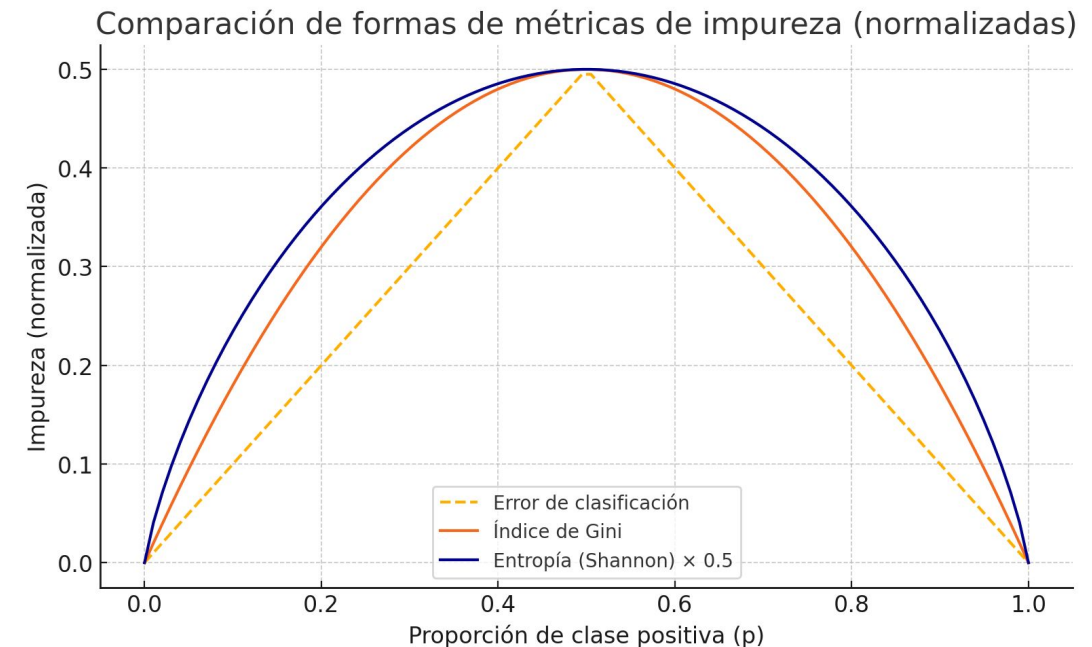
$$entropia = - \sum_{i=1}^n p_i \log_2(p_i)$$

Donde p_i es la proporción de la clase i en el nodo y n la cantidad de clases.

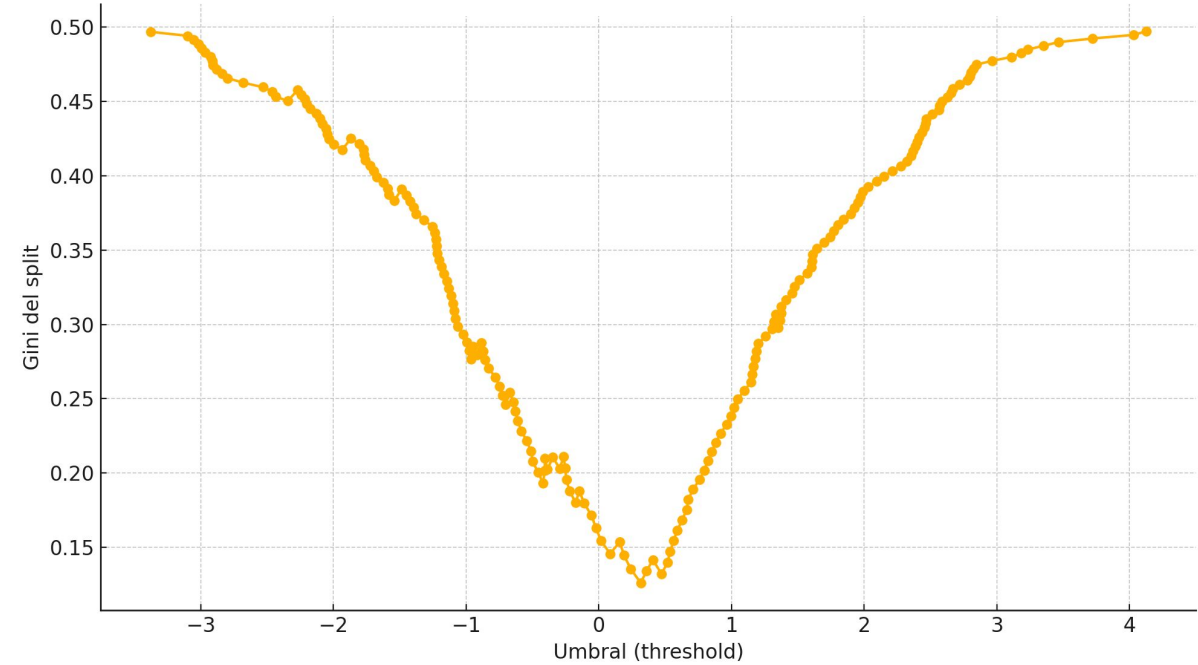
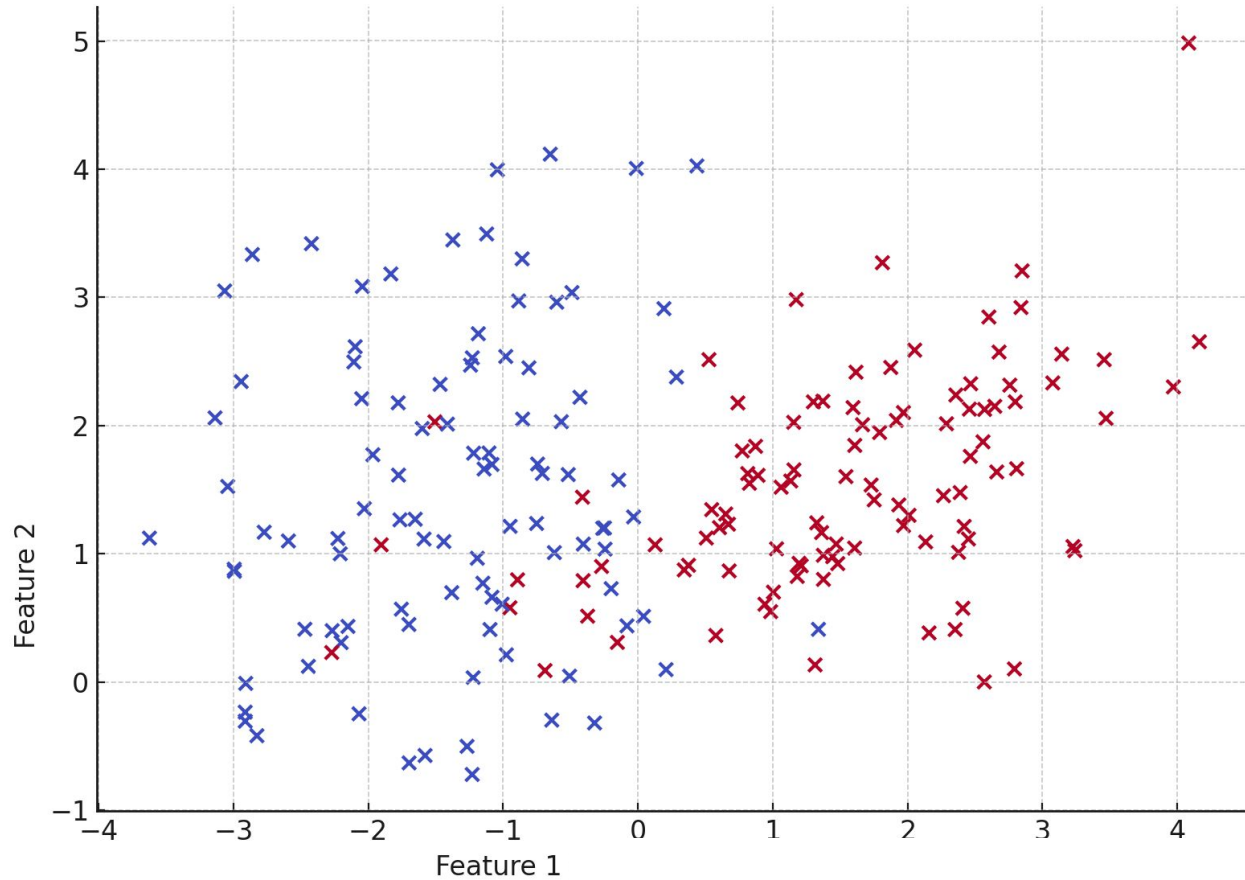
- Proviene de la teoría de la información (Claude Shannon, 1948):
 - mide **cantidad promedio de información** (o incertidumbre) contenida en un mensaje aleatorio.
 - Usada para cuantificar cuán impredecible es una fuente de información.
- Es aún más sensible a proporciones bajas (por su forma logarítmica).

Comparación de métricas

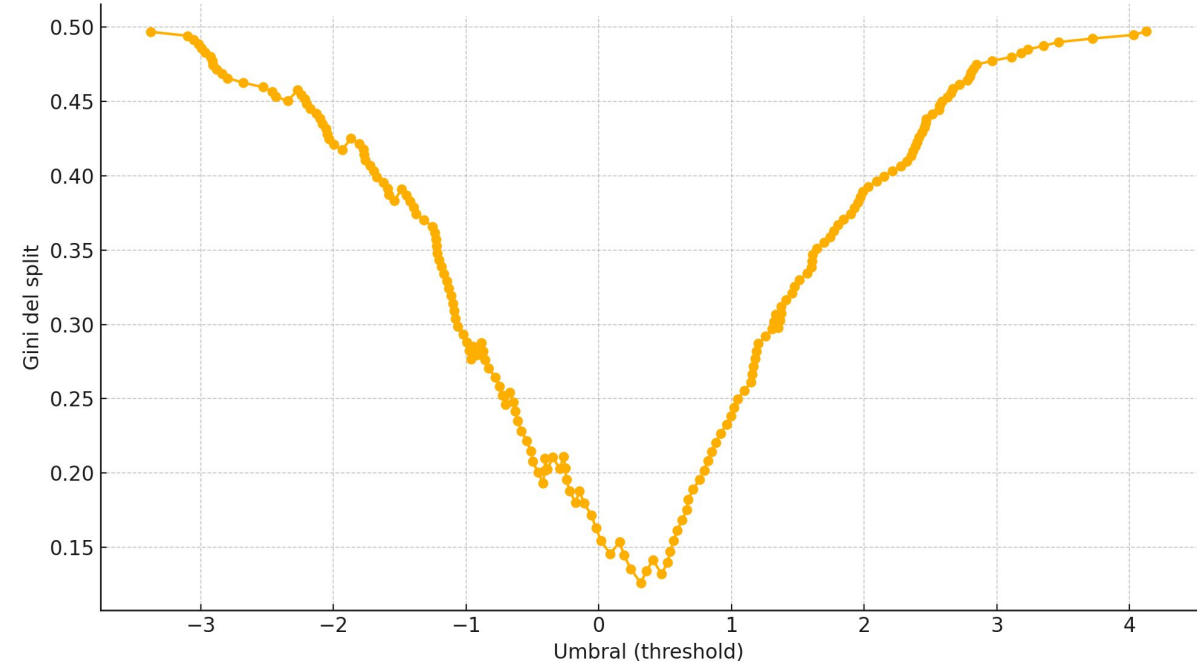
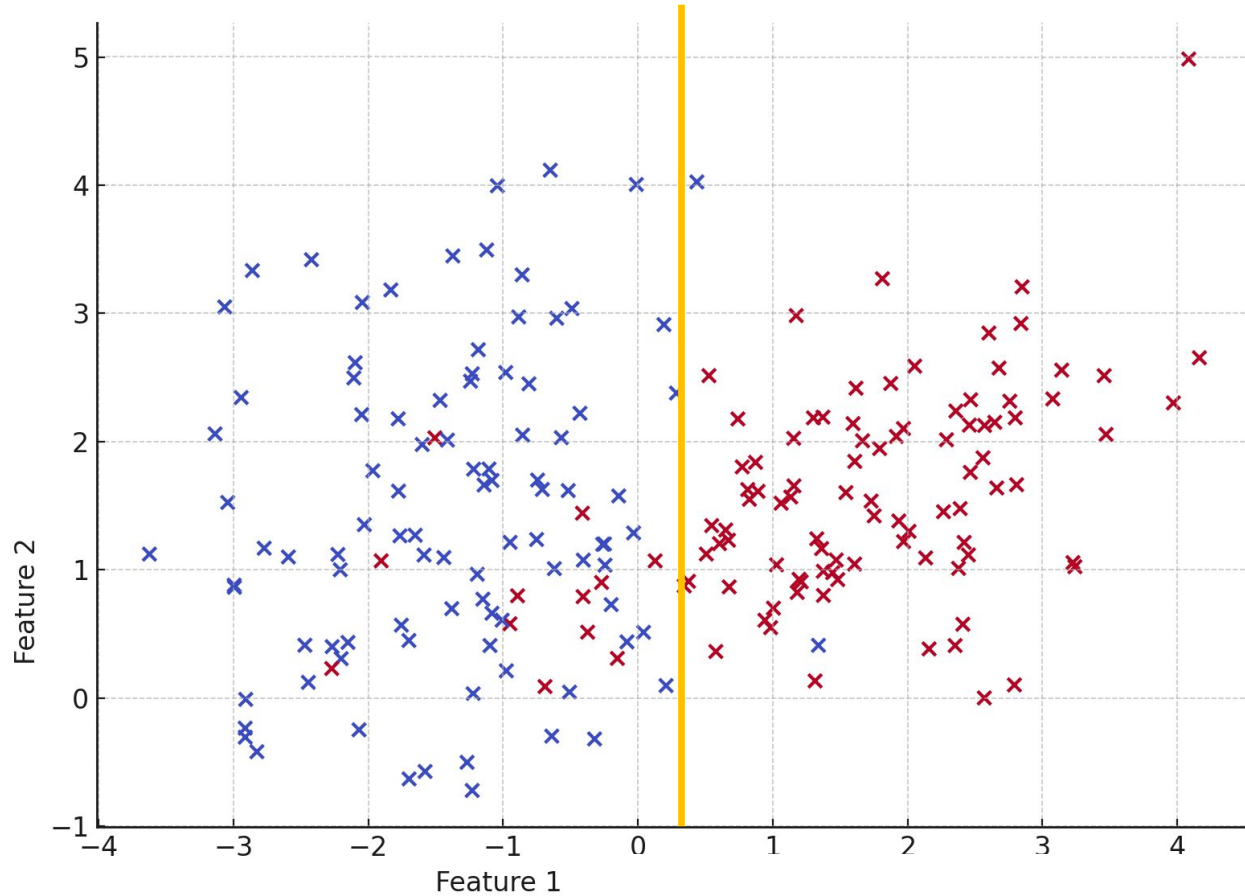
- El **error de clasificación** constante para todo p
- El **índice de Gini** y la **entropía** penalizan más los errores cuando hay pocas muestras de una clase. (p pequeño)
- **Entropía y Gini** mas sensibles a cambios en pequeñas proporciones les permite construir **divisiones más informativas**.
- *Se suele usar Gini ya que es computacionalmente mas eficiente que Shannon (multiplicación vs logaritmo)



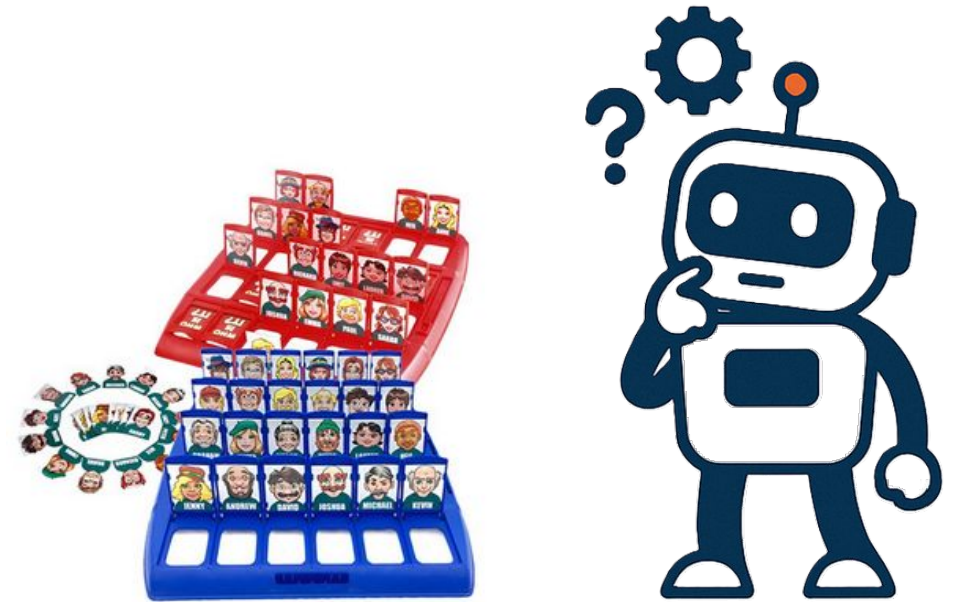
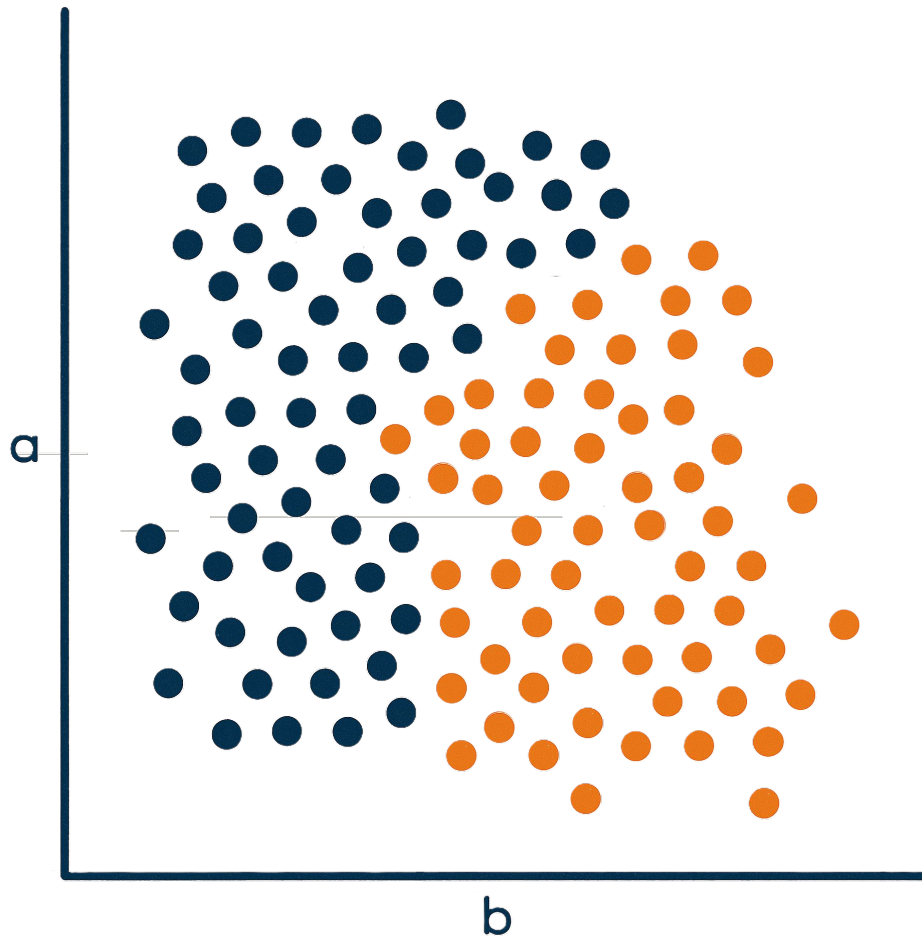
Ejemplo de calculo de Gini



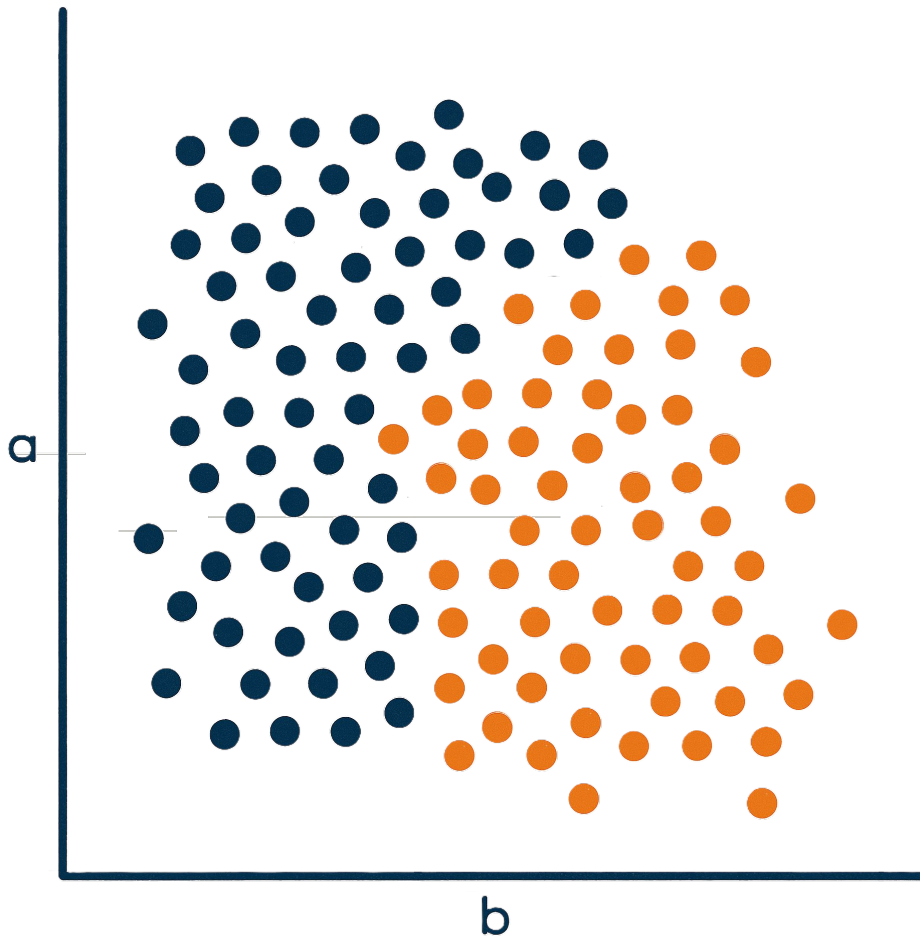
Ejemplo de calculo de Gini



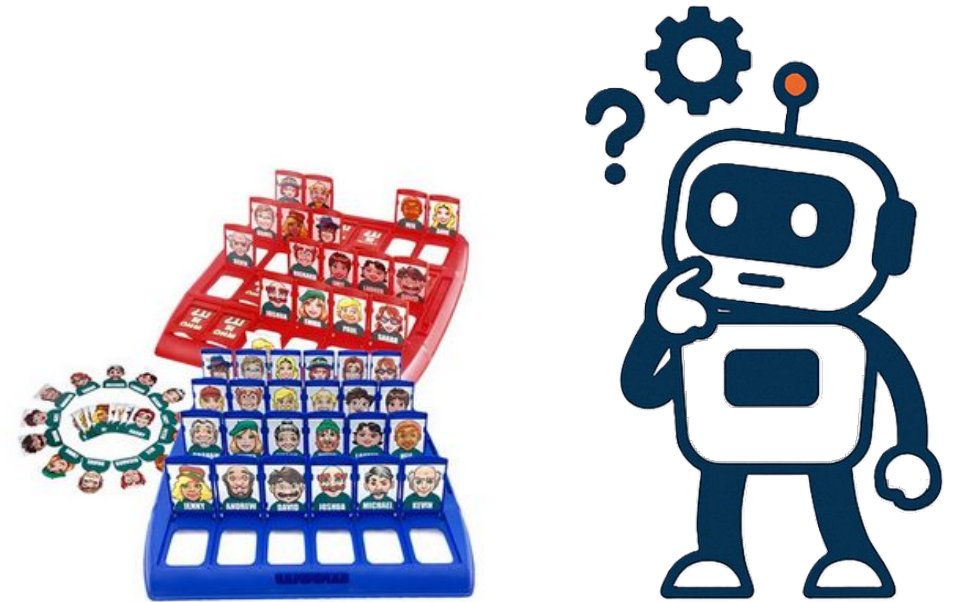
¿Cómo se construye un Árbol de Decisión?



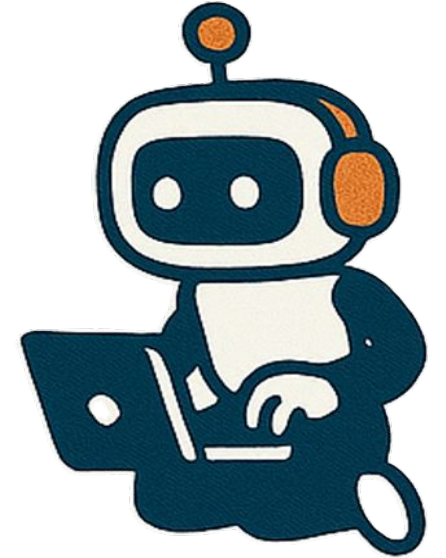
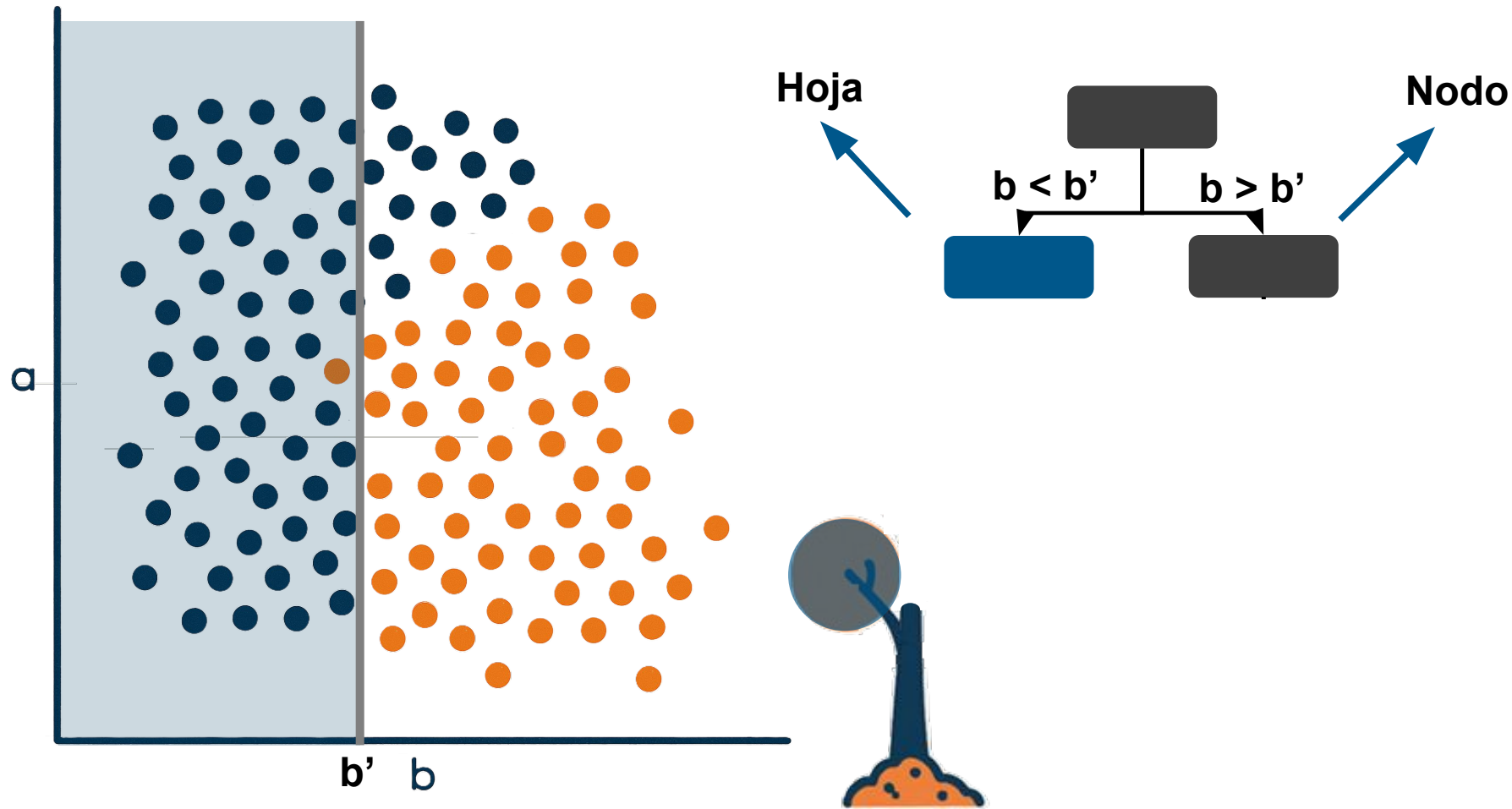
¿Cómo se construye un Árbol de Decisión?



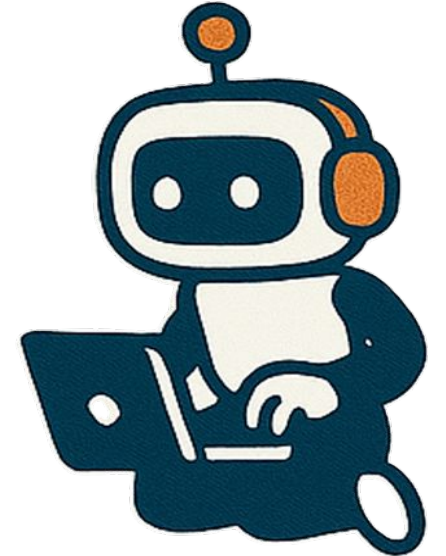
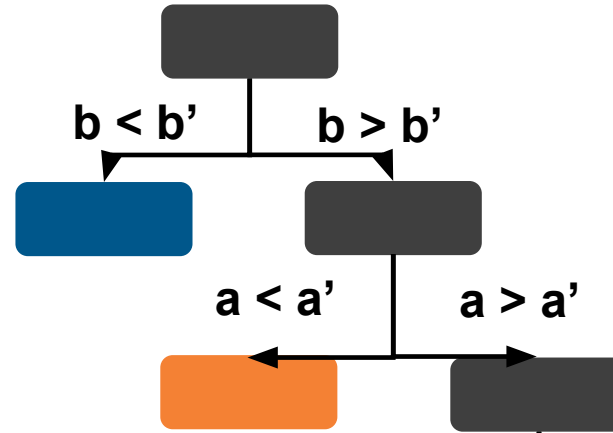
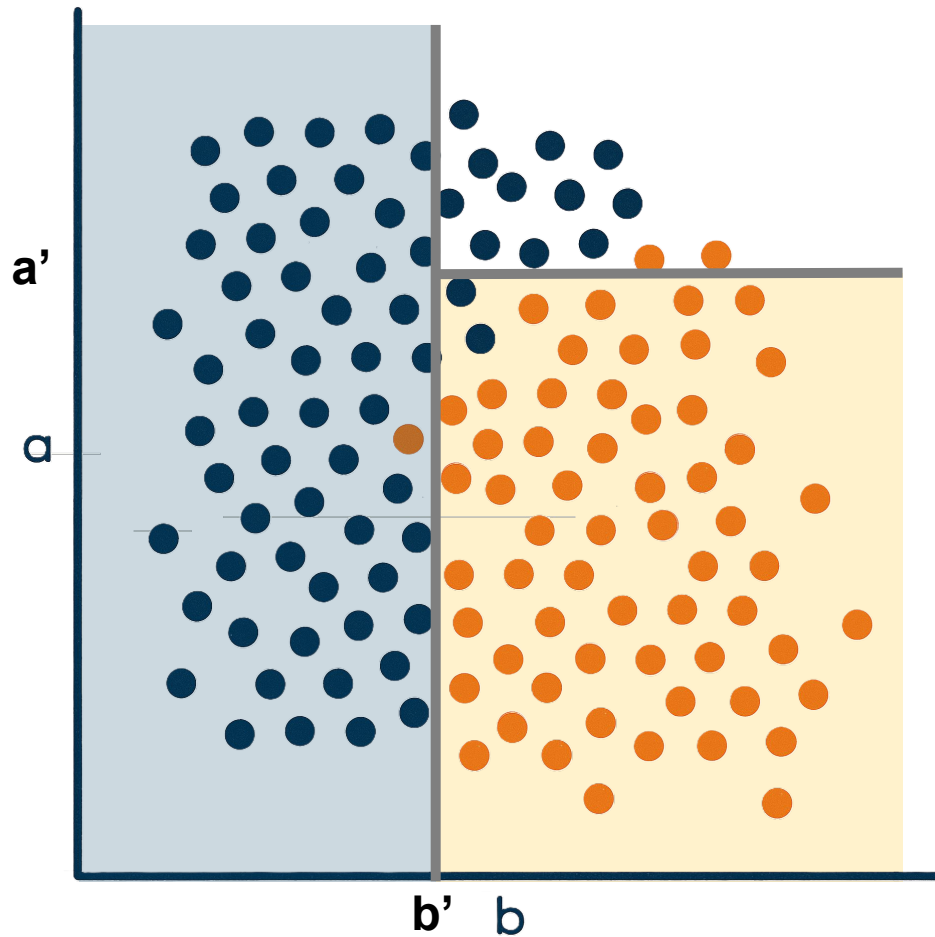
¡Ya sabemos elegir un umbral!



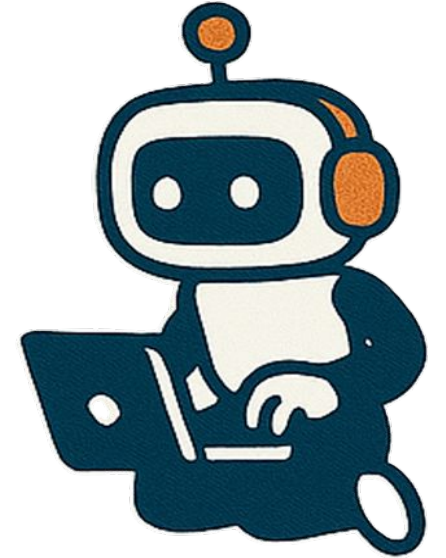
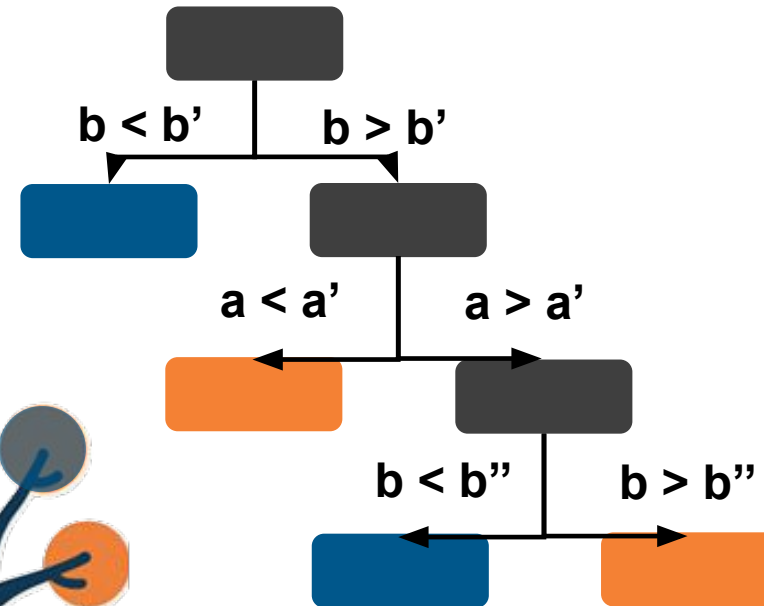
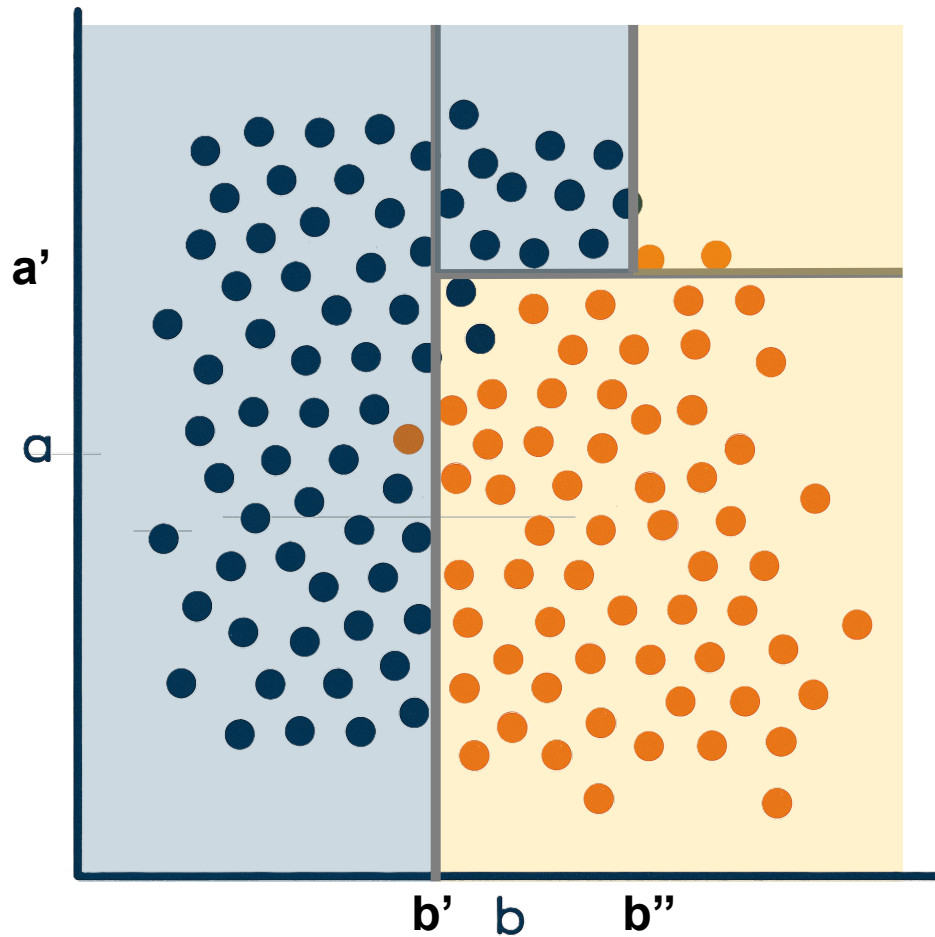
¿Cómo se construye un Árbol de Decisión?



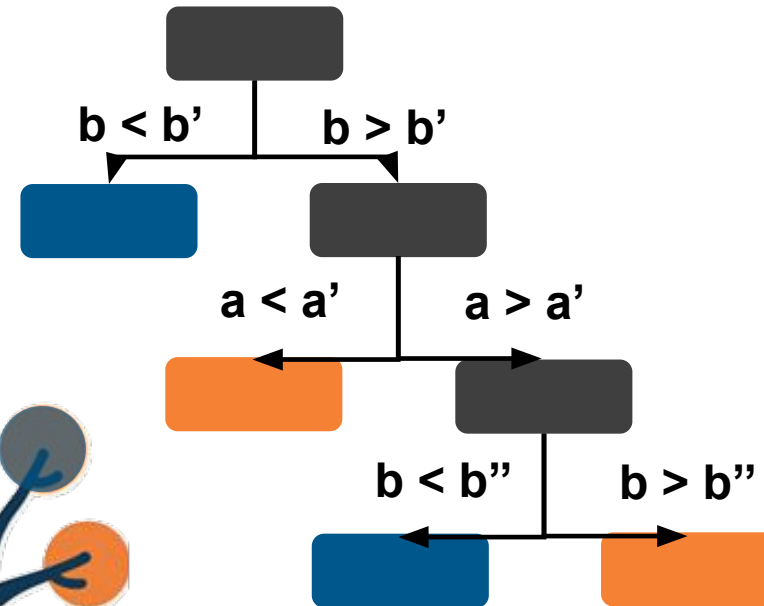
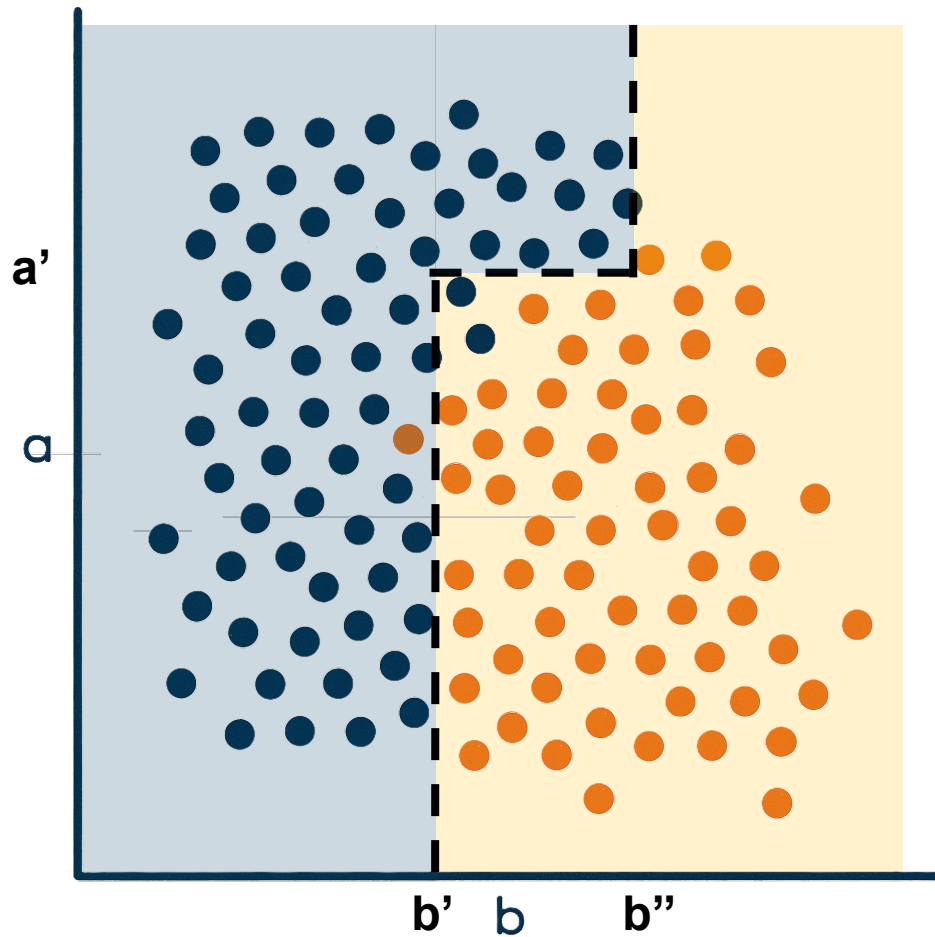
¿Cómo se construye un Árbol de Decisión?



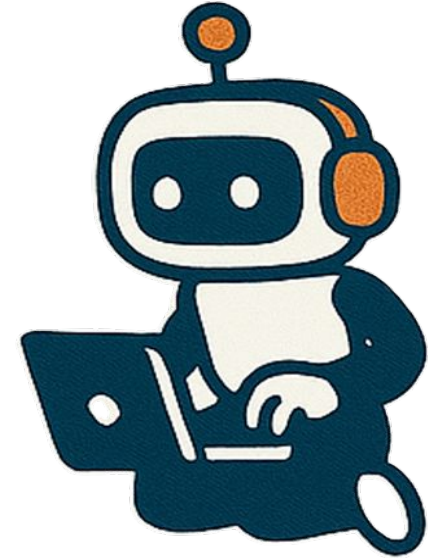
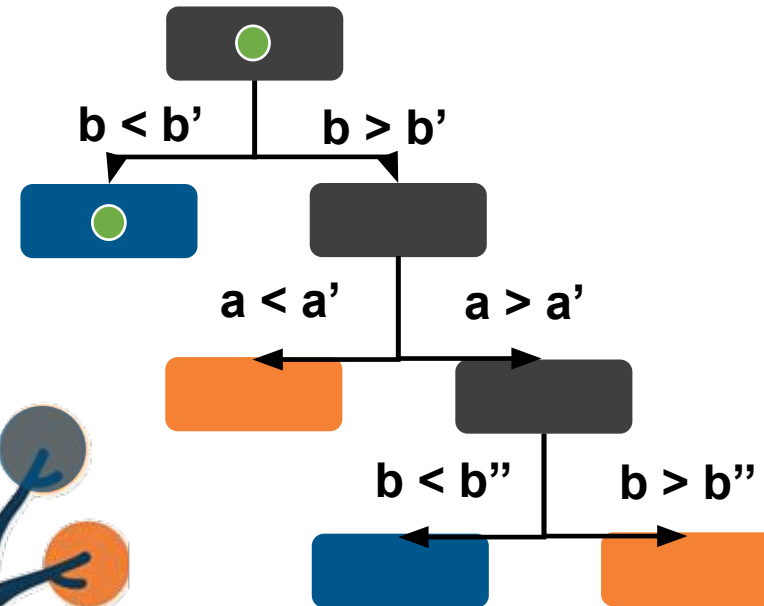
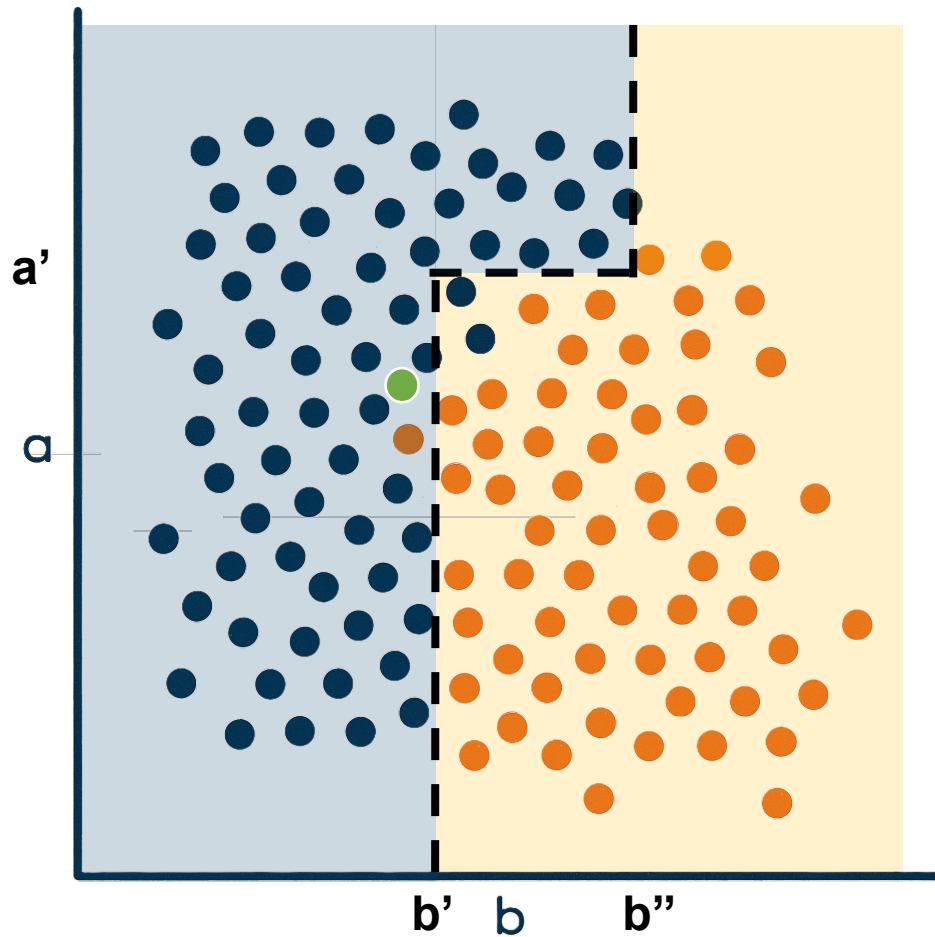
¿Cómo se construye un Árbol de Decisión?



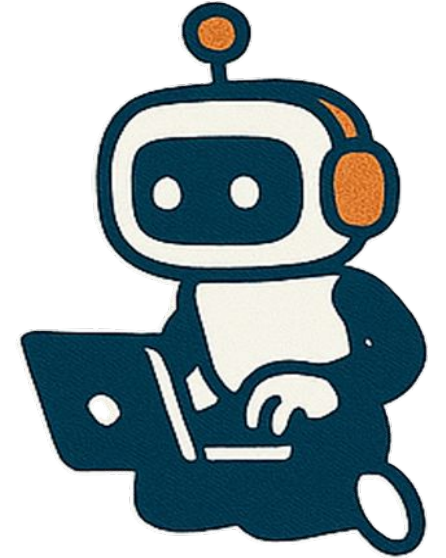
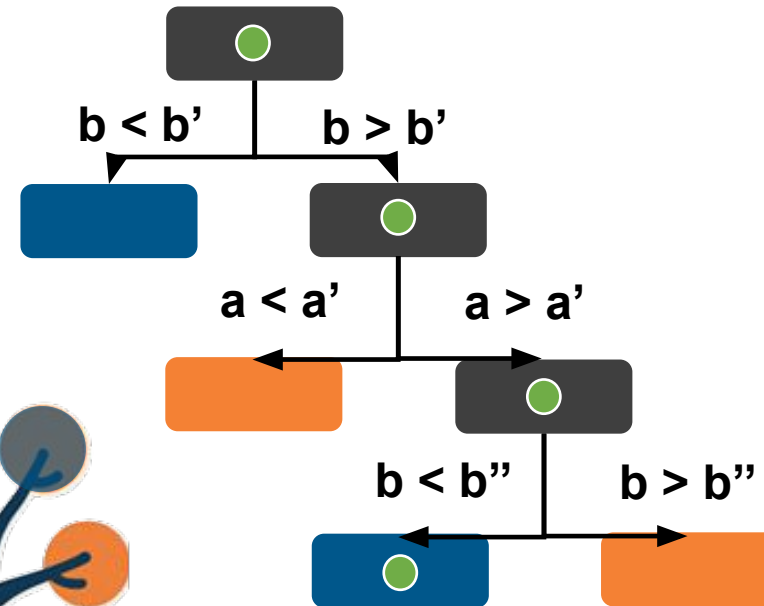
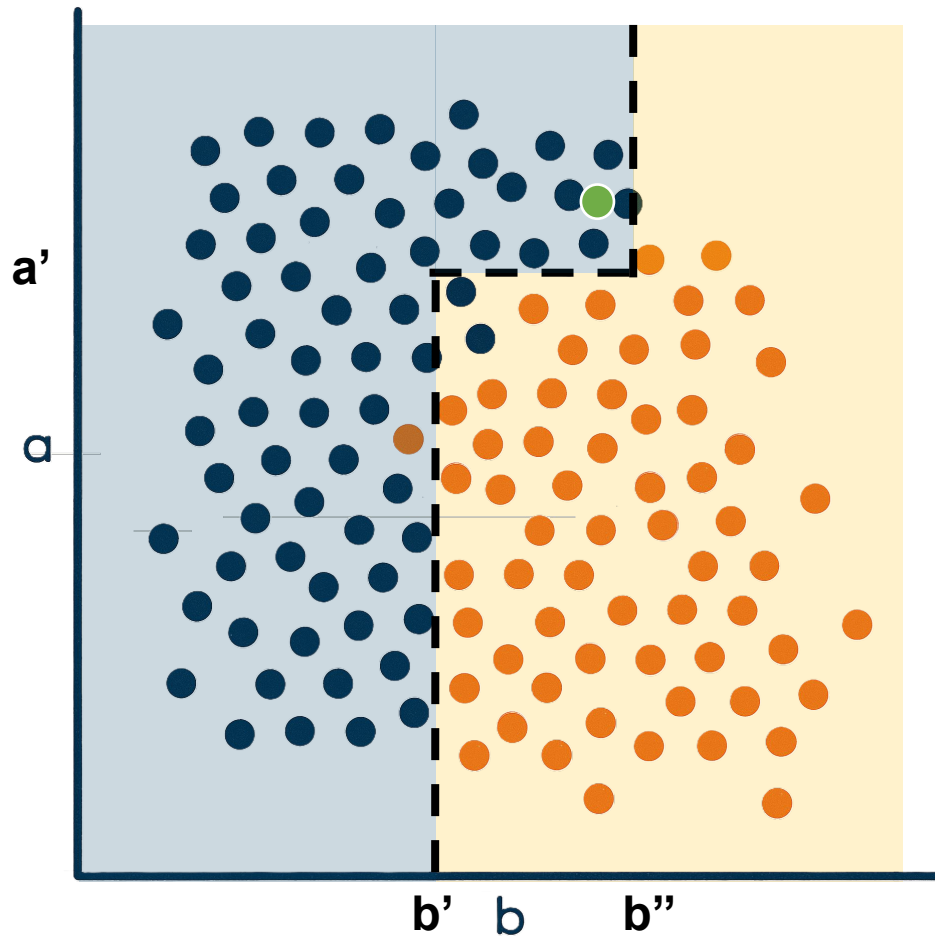
¿Cómo se construye un Árbol de Decisión?



¿Cómo predice un Árbol de Decisión?

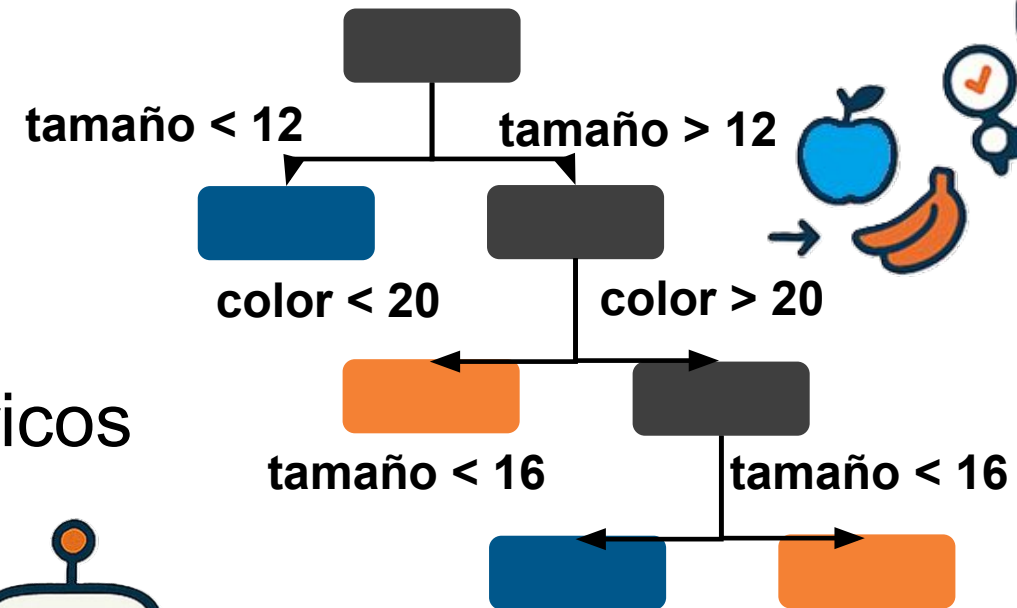
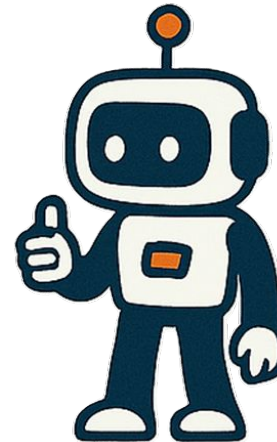


¿Cómo predice un Árbol de Decisión?



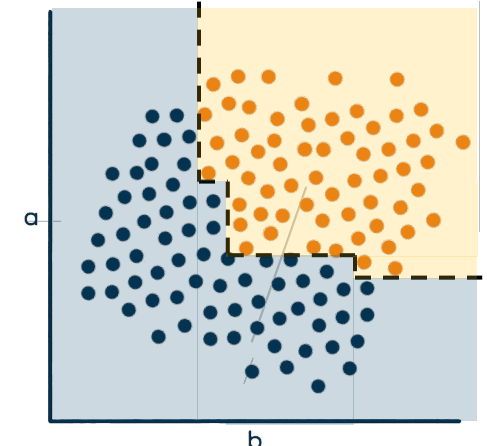
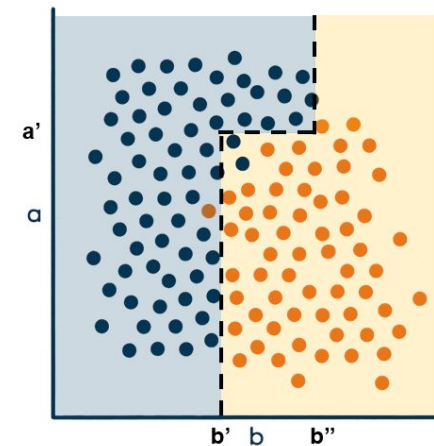
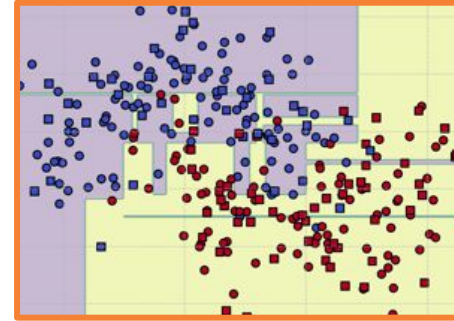
Ventajas de Árboles de Decisión

- Interpretabilidad clara
- Maneja datos numéricos y categóricos
- No necesita normalización

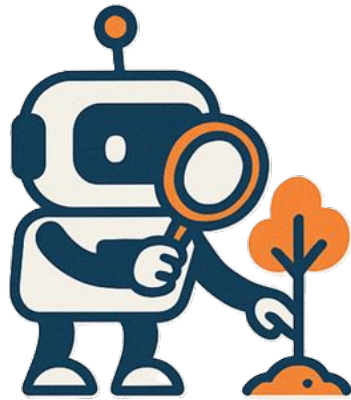


Desventajas de Árboles de Decisión

- Propenso al sobreajuste
- Fronteras de decisión muy rígidas
- Vulnerable a la orientación de las features
- Sensible a pequeñas variaciones
 - Pequeño cambio en datos puede dar un árbol muy distinto

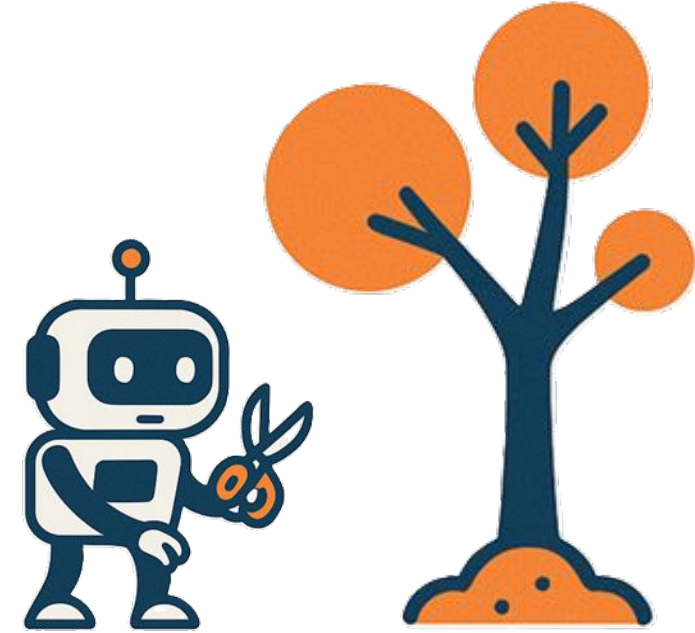


Técnicas para controlar sobreajuste



Restricciones durante el crecimiento

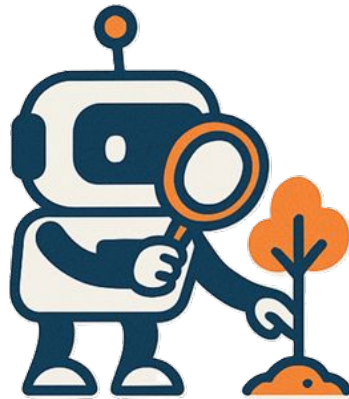
- Limitar la profundidad del árbol
- Evitar que cree un nodo si tiene pocos datos.
- Imponer mejora mínima para crear un nodo.



Poda

- 1) Se hace crecer el árbol sin restricciones
- 2) Eliminar ramas que aportan poca mejora

Técnicas para controlar sobreajuste



Generalmente se usa la pre-poda ya que es mas eficiente computacionalmente



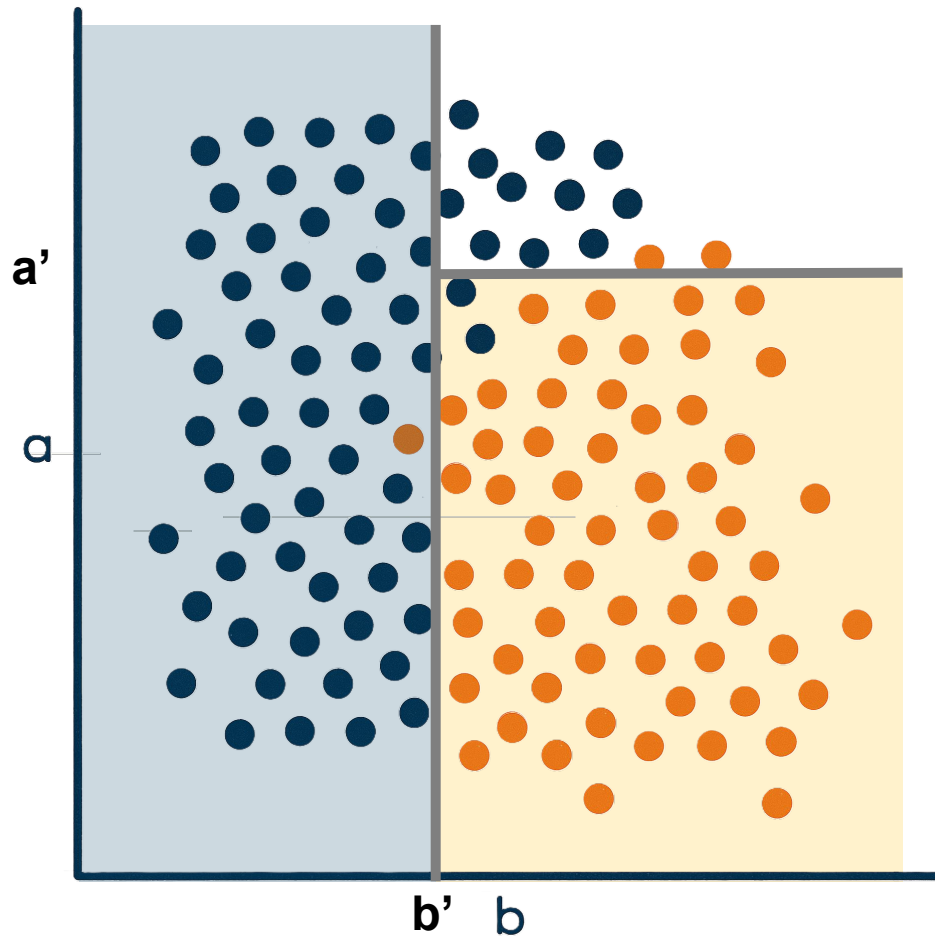
Restricciones durante el crecimiento

- Limitar la profundidad del árbol
- Evitar que cree un nodo si tiene pocos datos.
- Mejora mínima para crear un nodo.

Poda

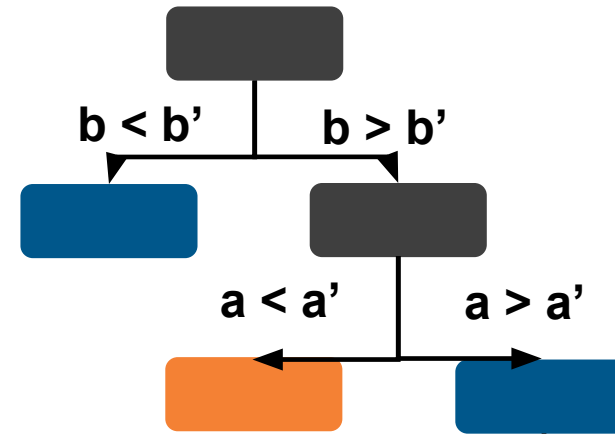
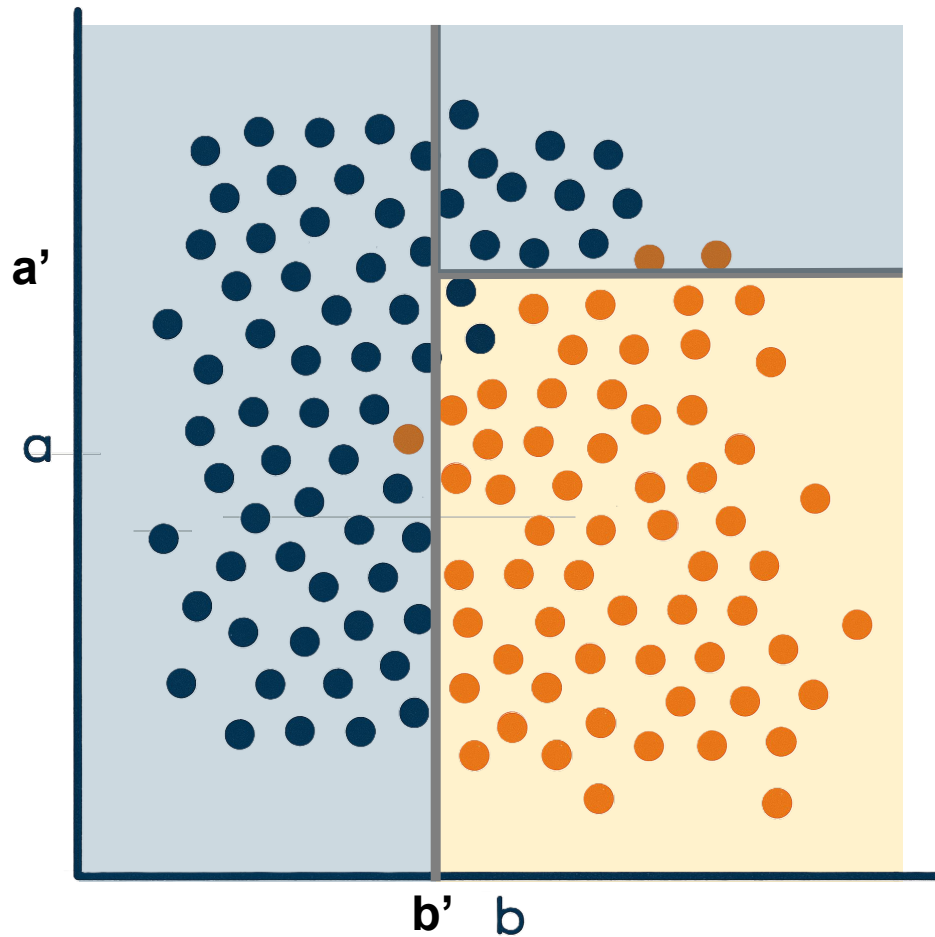
- 1) Se hace crecer el árbol sin restricciones
- 2) Eliminar ramas que aportan poca mejora

¿Cómo aplican restricciones al crecimiento?



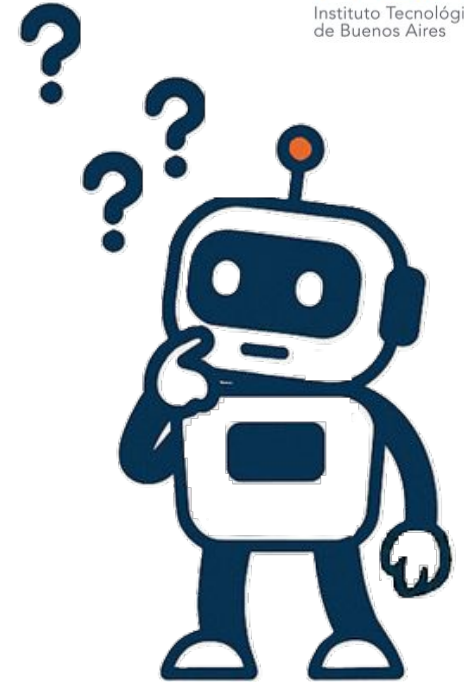
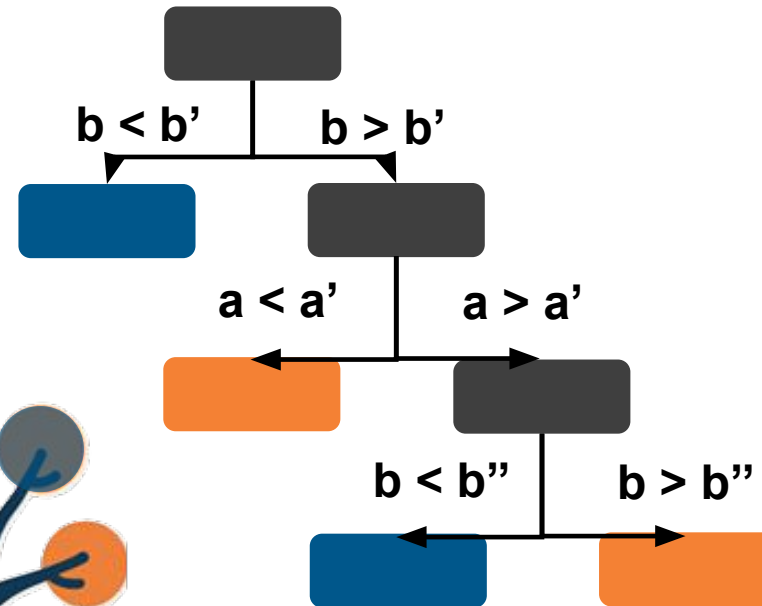
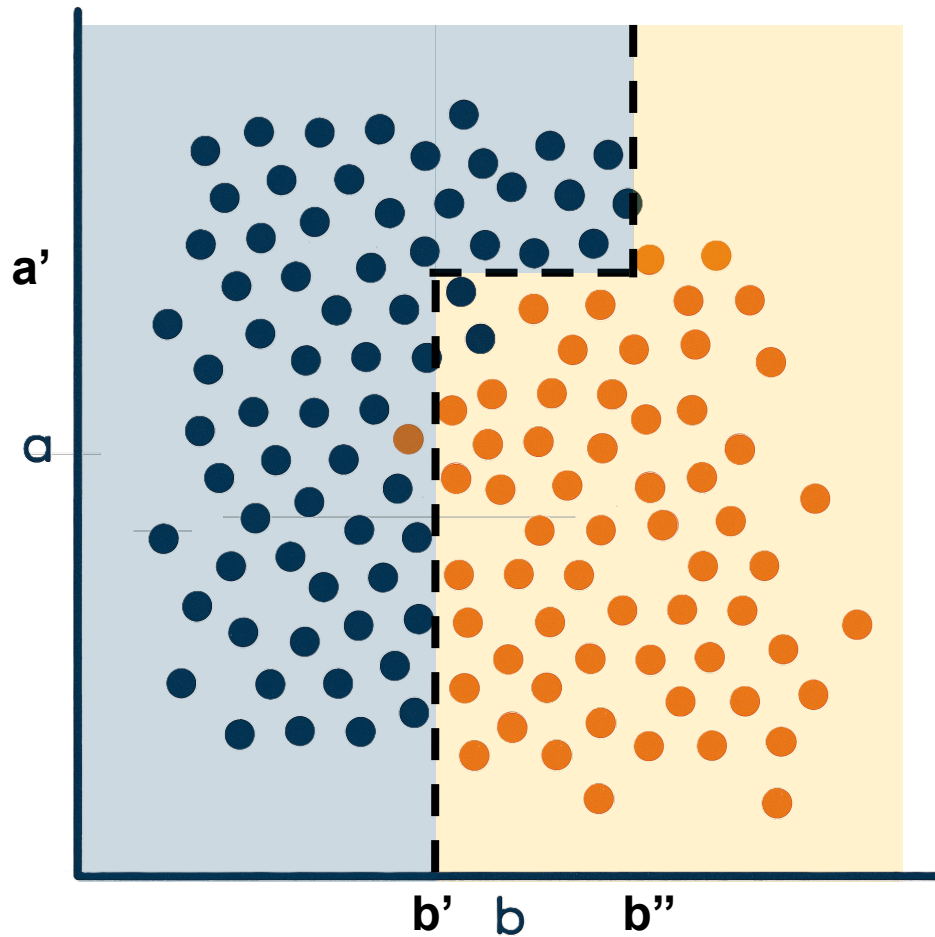
Se crea una hoja, de la clase mayoritaria en el nodo

¿Cómo se construye un Árbol de Decisión?



Se crea una hoja, de la clase mayoritaria en el nodo

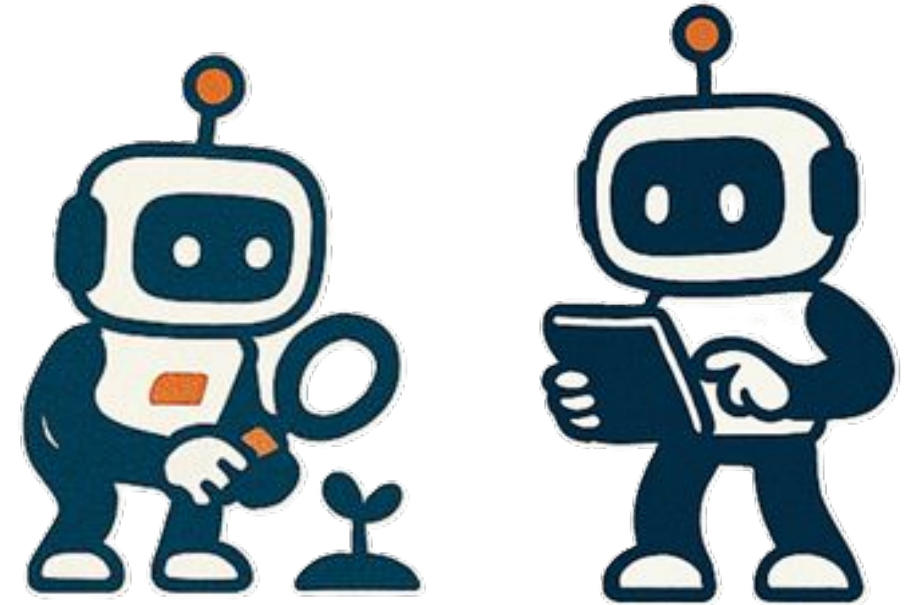
¿dudas hasta aquí?



¿Qué sabemos hasta ahora?

1. Como funciona un árbol de decisión
2. Evaluar los distintos umbrales posibles
3. Técnicas para evitar overfitting

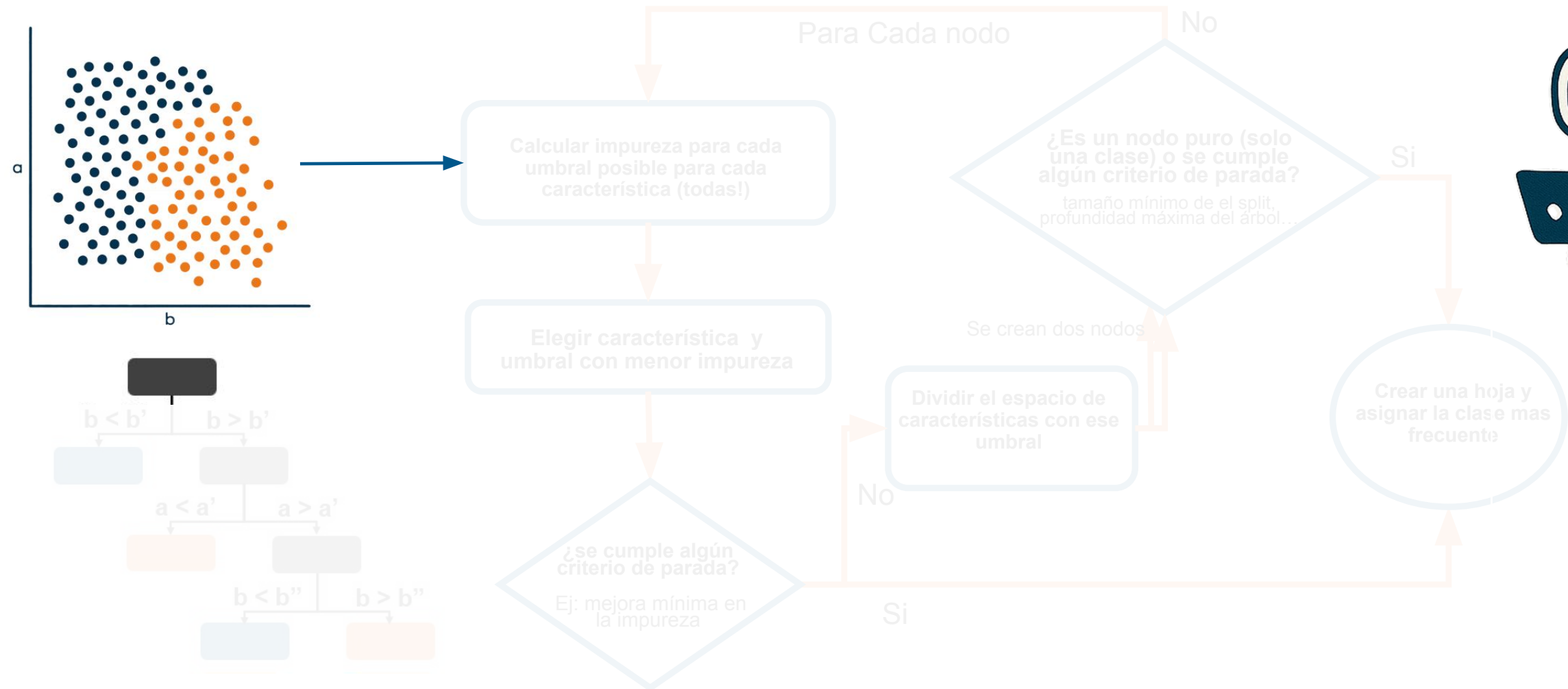
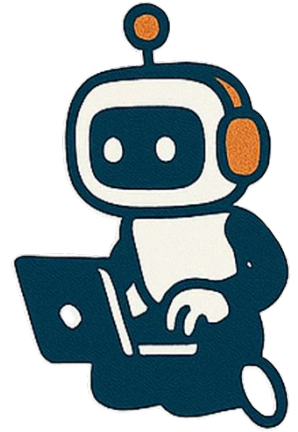
Ahora... ¿como se construye?



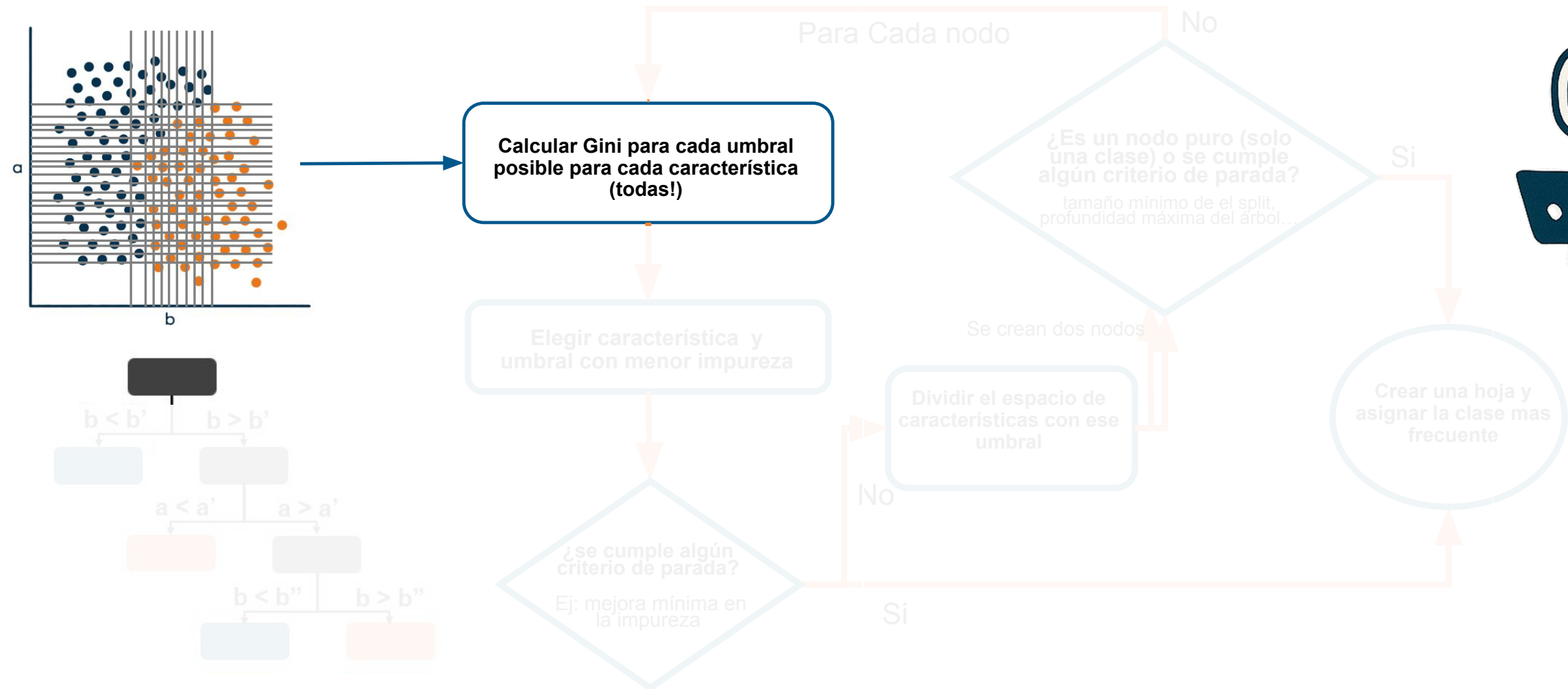
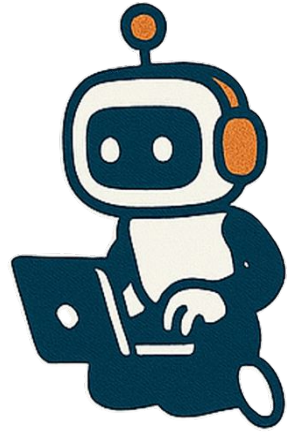
¿Como construimos un árbol?

- Históricamente se han propuesto varios algoritmos, como ID3 y C4.5,
- En la actualidad el más utilizado es CART:
 - Más simple e intuitivo
 - Tiene amplio soporte en librerías modernas (como scikit-learn, Random Forest y XGBoost...)

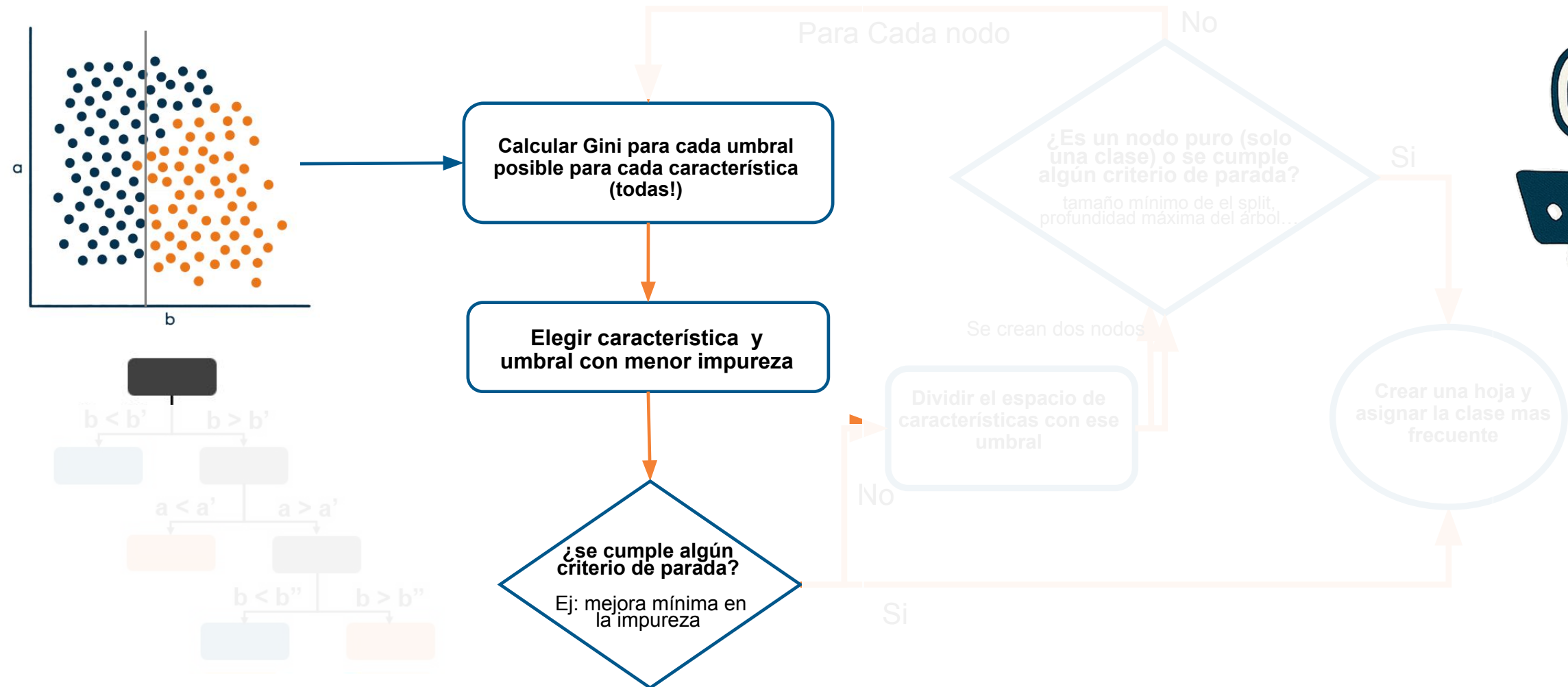
¿Como funciona el algoritmo CART?



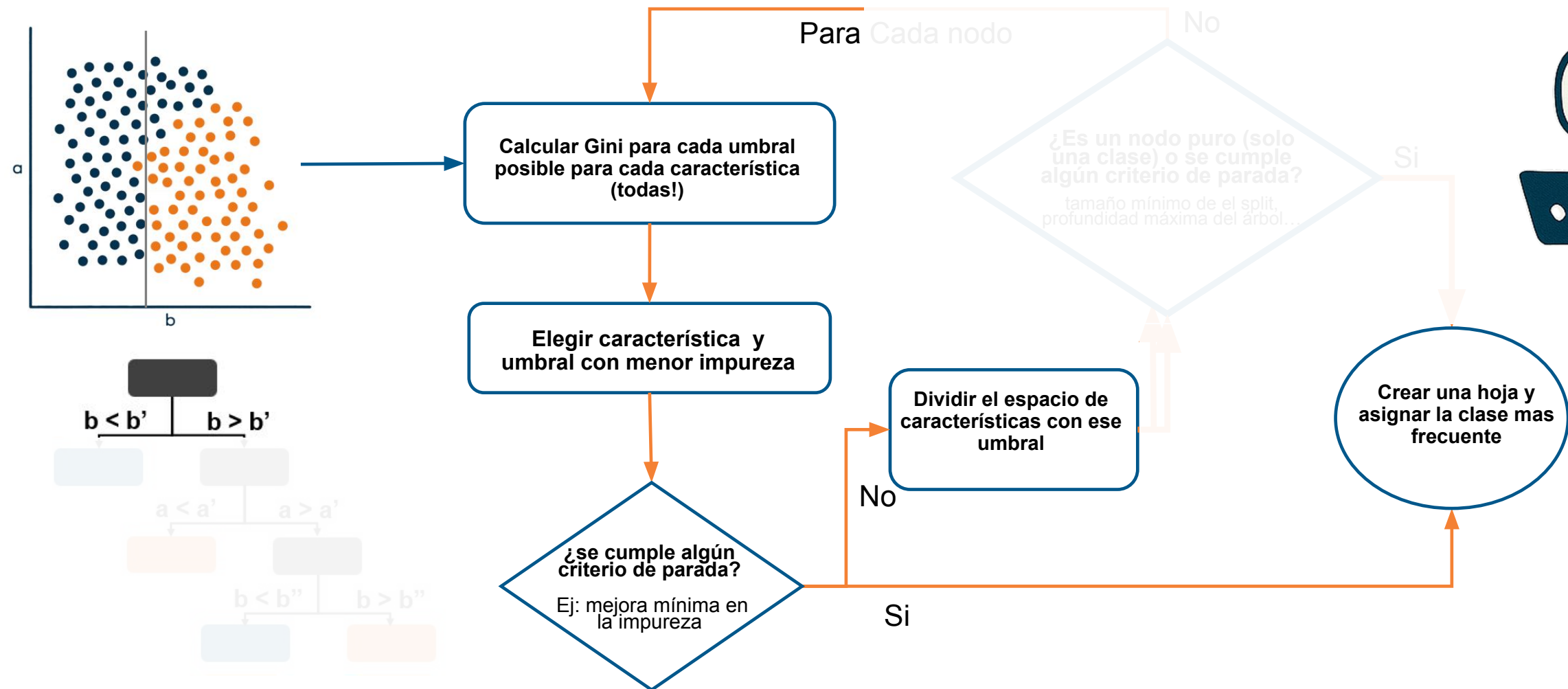
¿Como funciona el algoritmo CART?



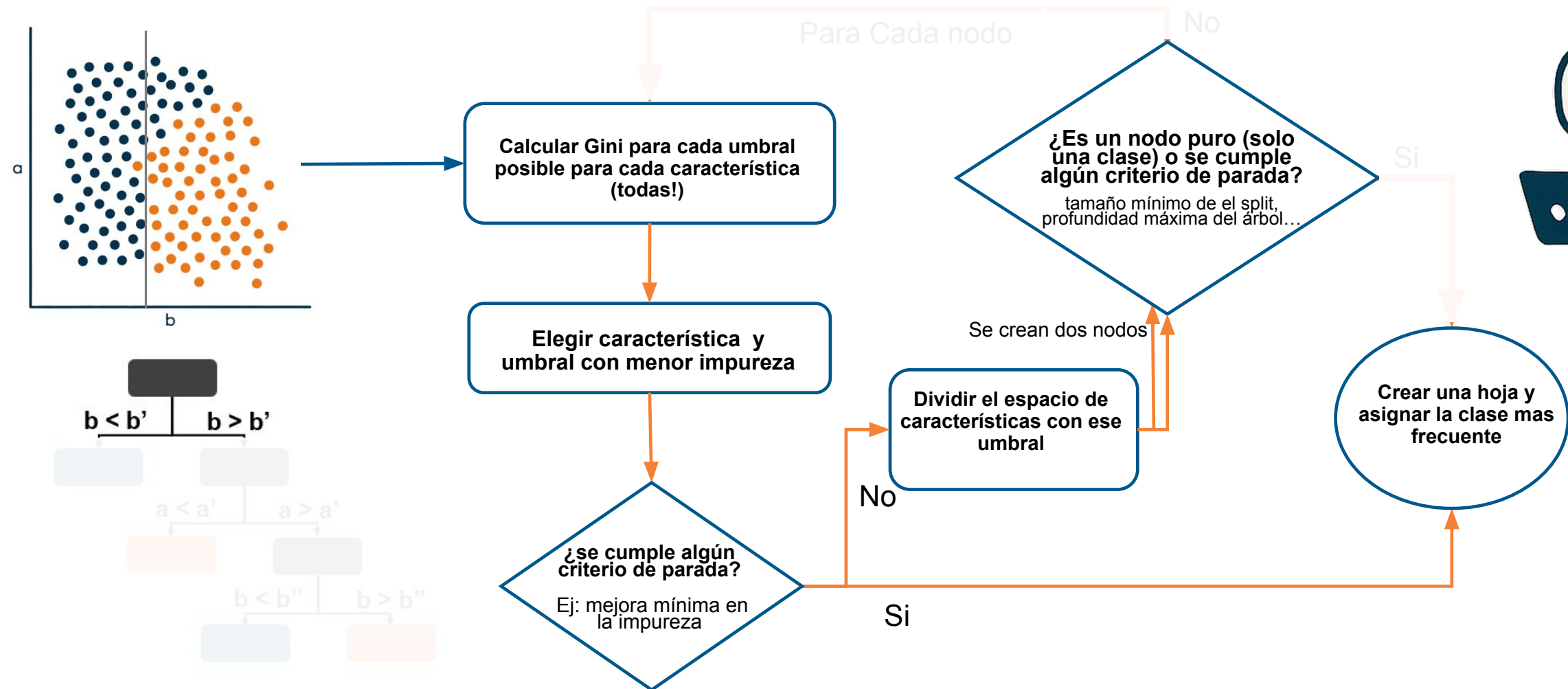
¿Como funciona el algoritmo CART?



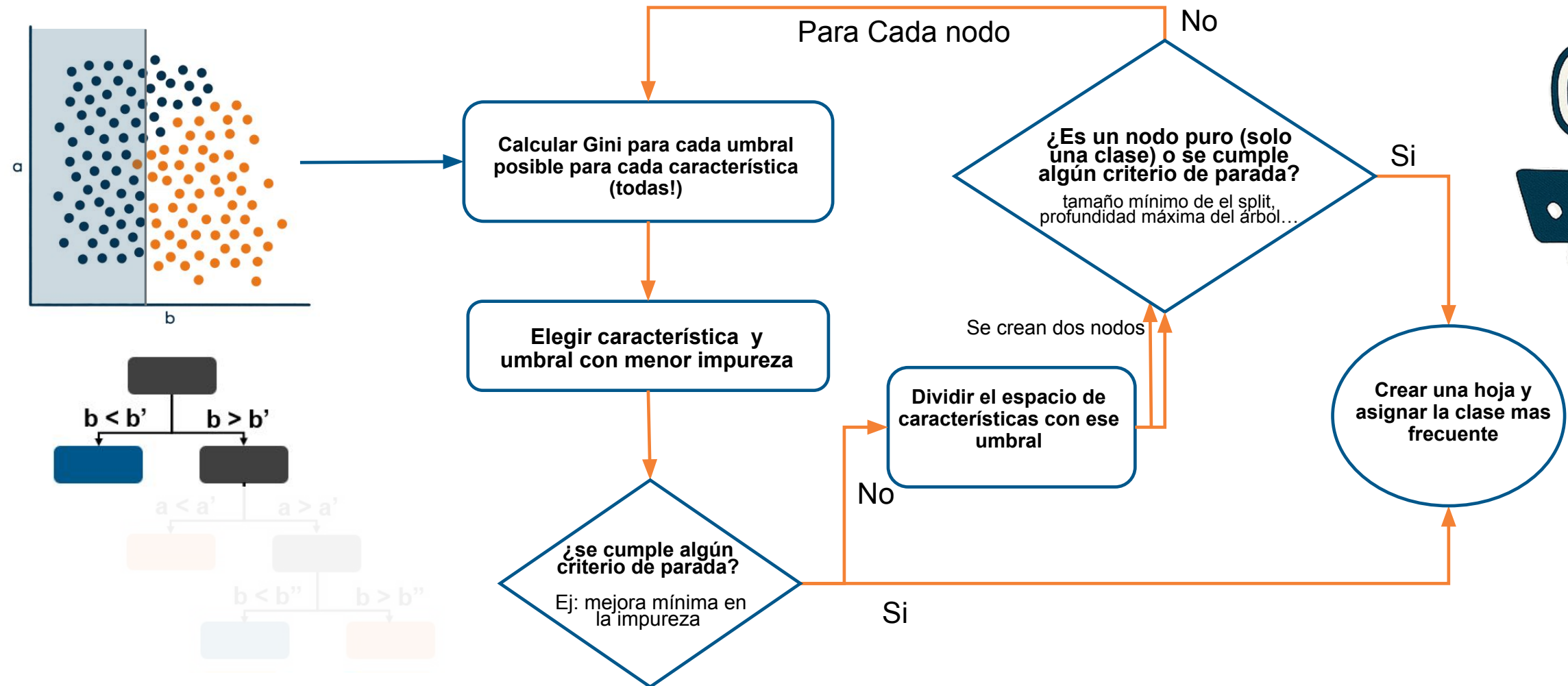
¿Como funciona el algoritmo CART?



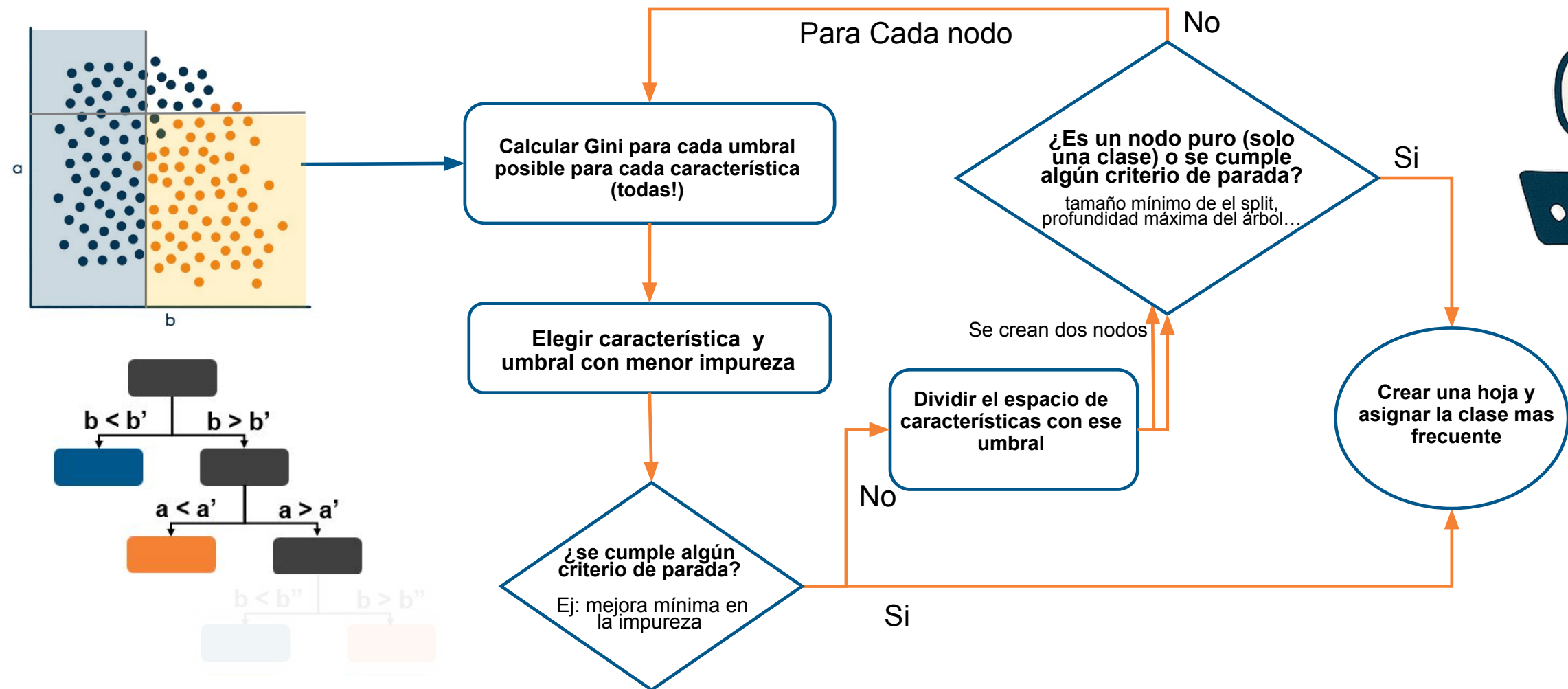
¿Como funciona el algoritmo CART?



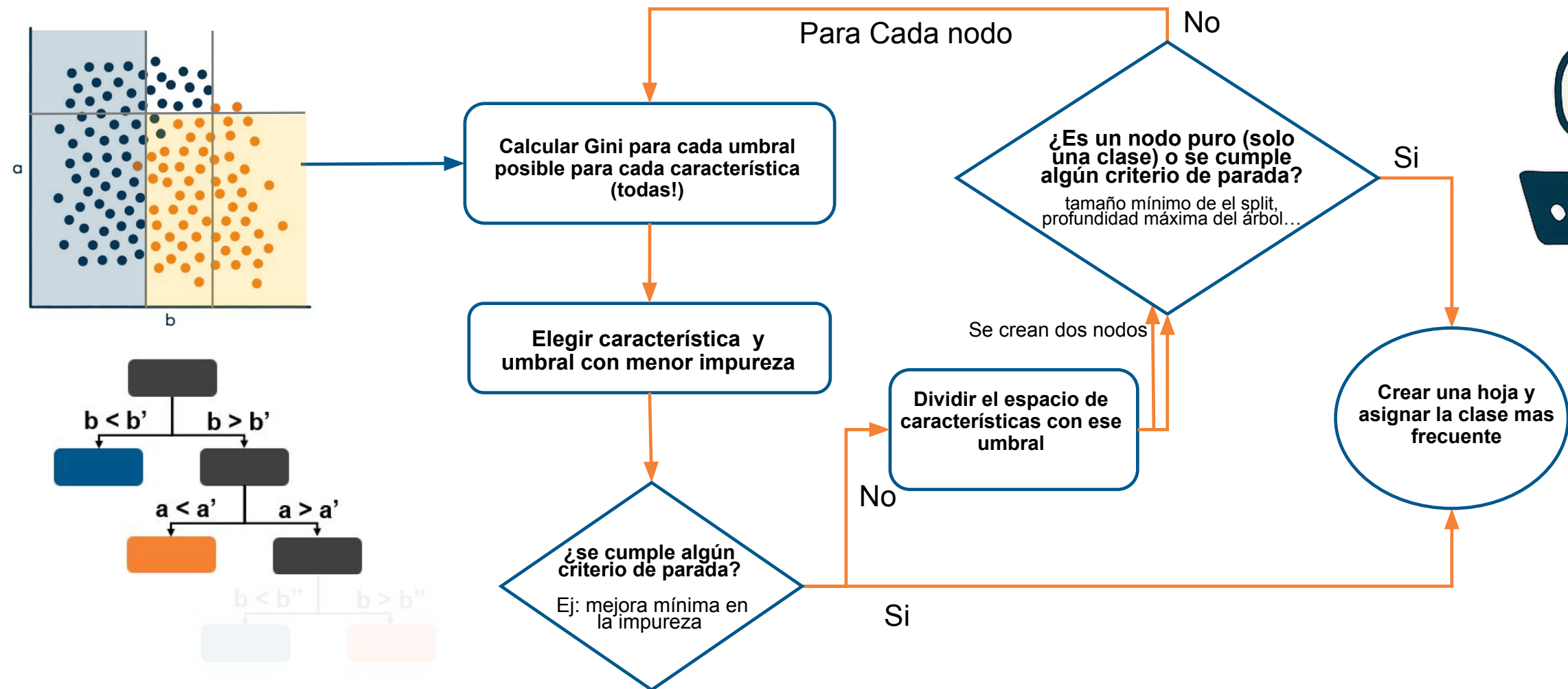
¿Como funciona el algoritmo CART?



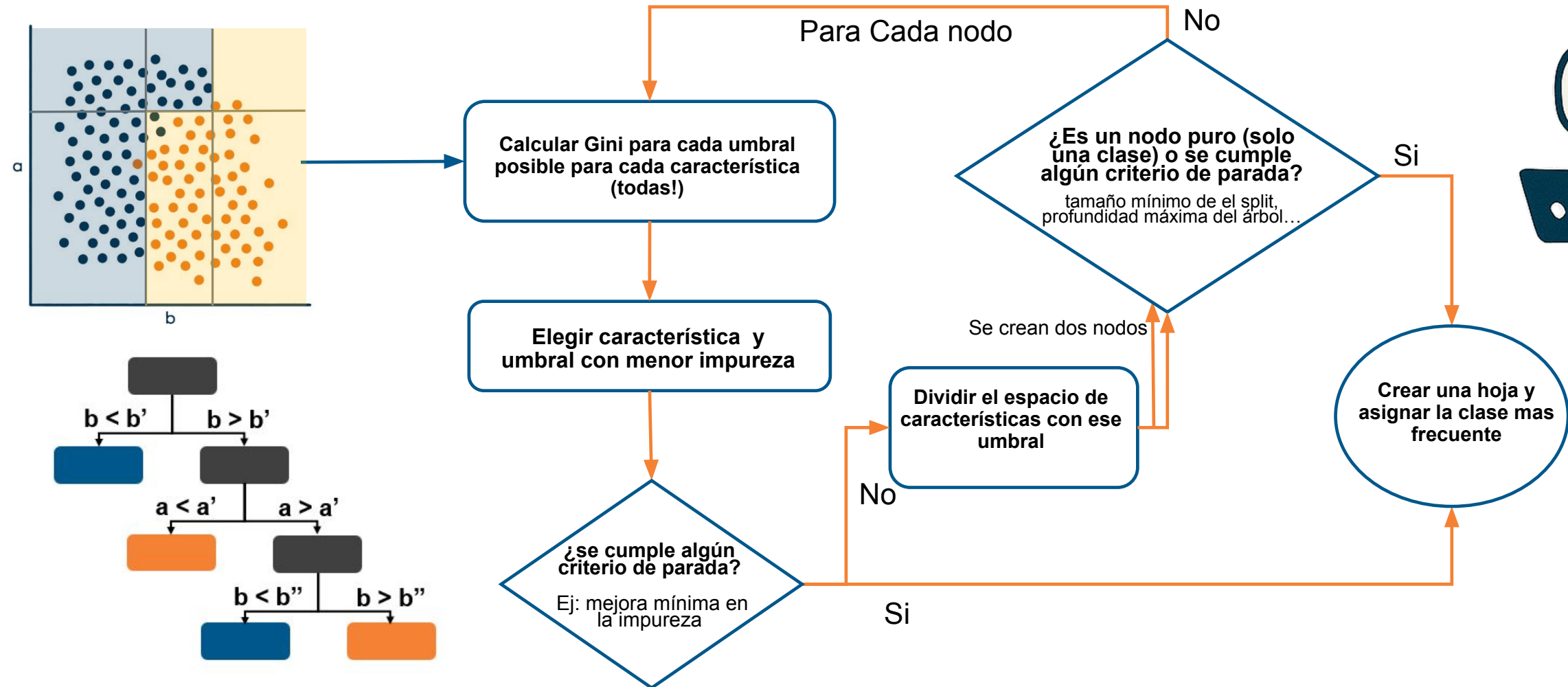
¿Como funciona el algoritmo CART?



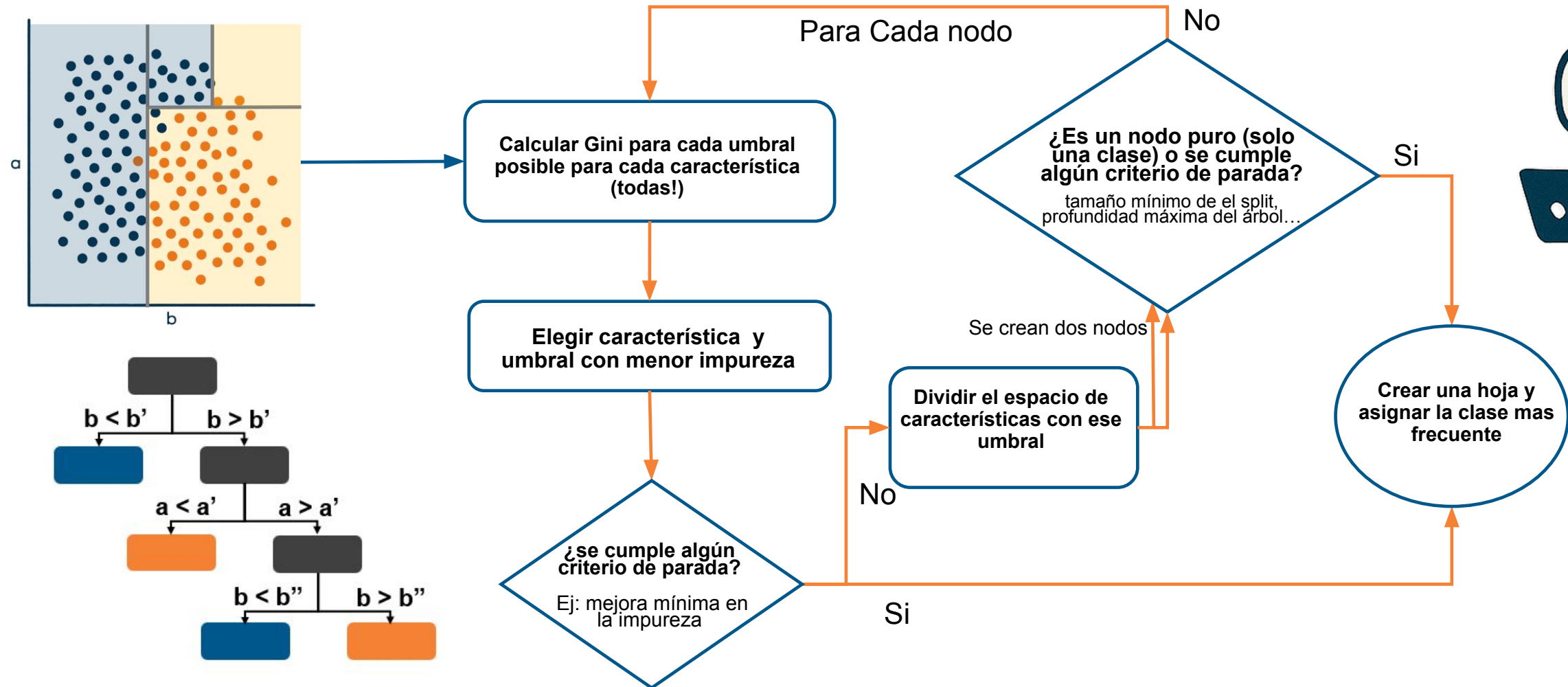
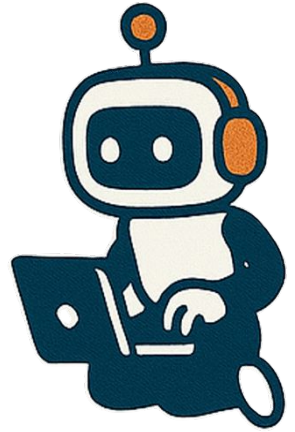
¿Como funciona el algoritmo CART?



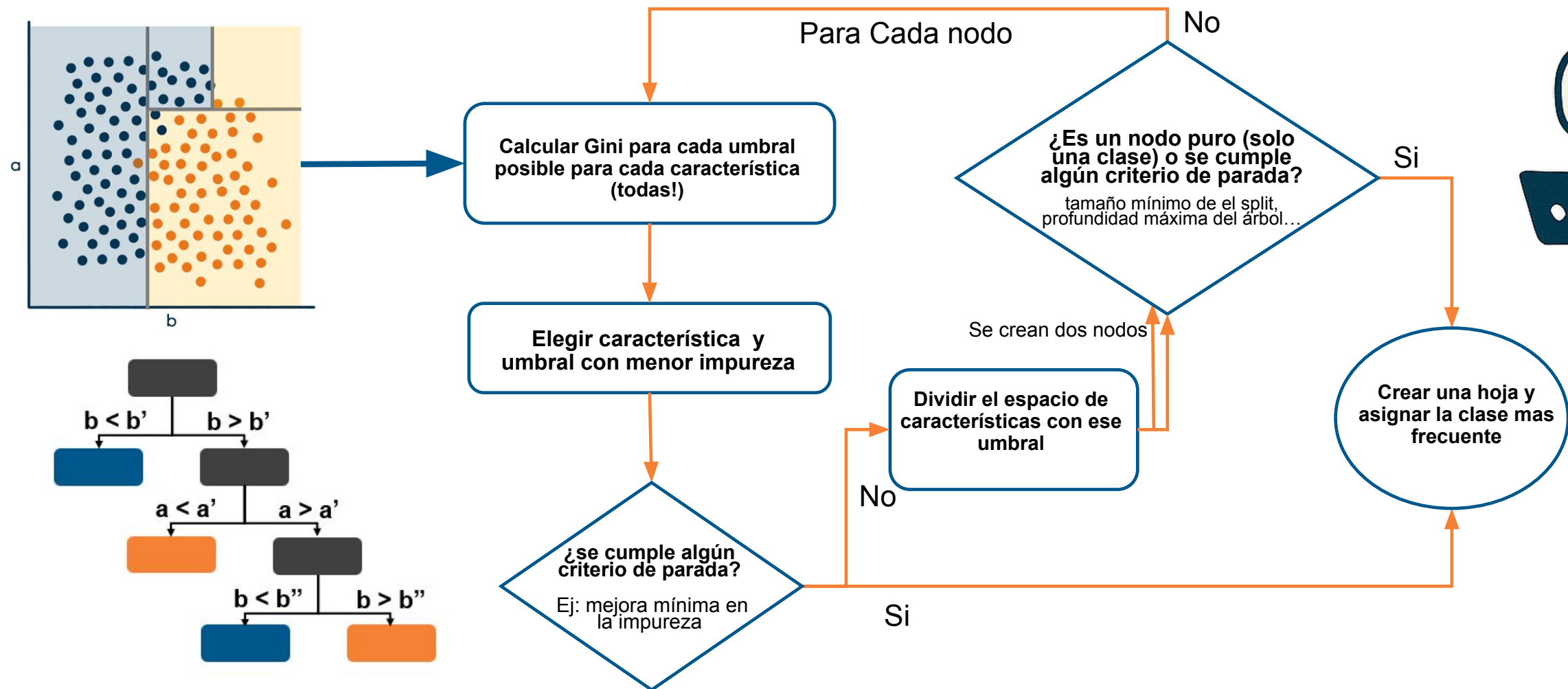
¿Como funciona el algoritmo CART?



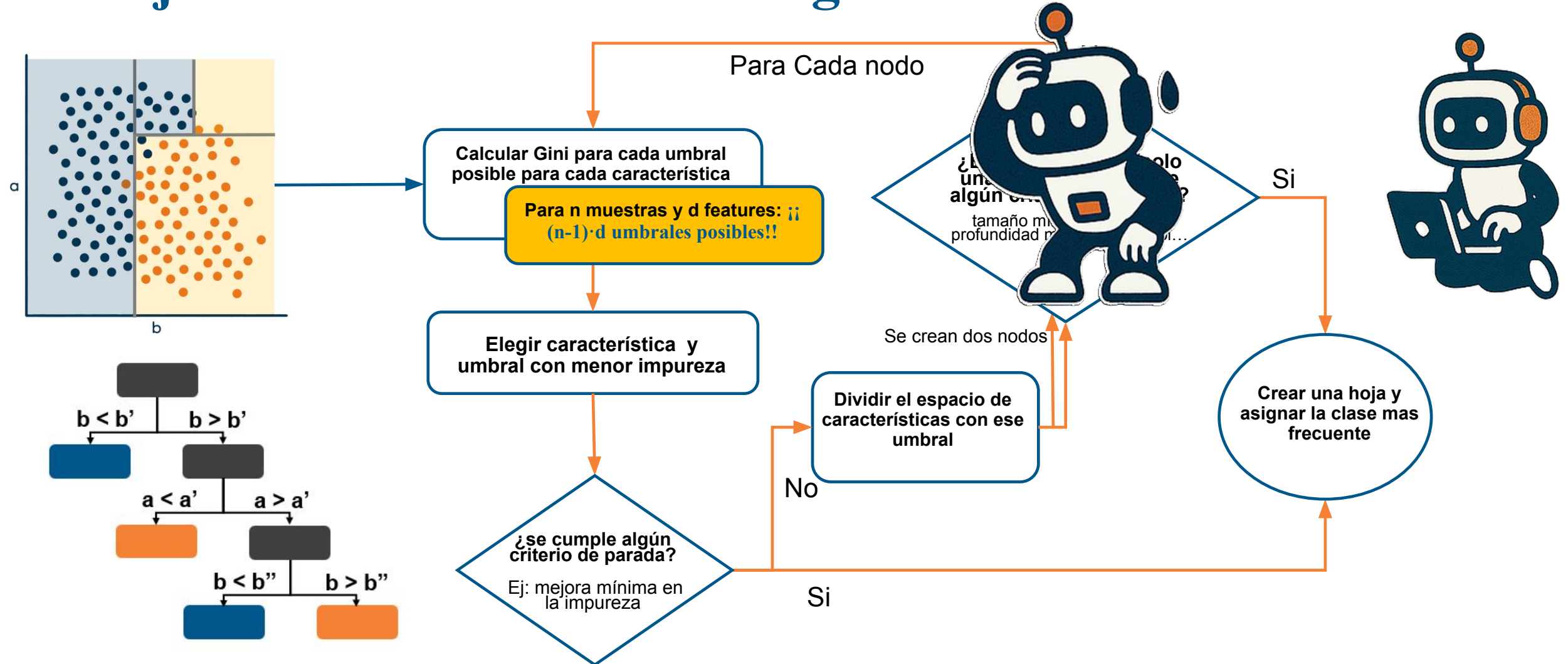
¿Como funciona el algoritmo CART?



¿En que parte podemos mejorar la eficiencia?



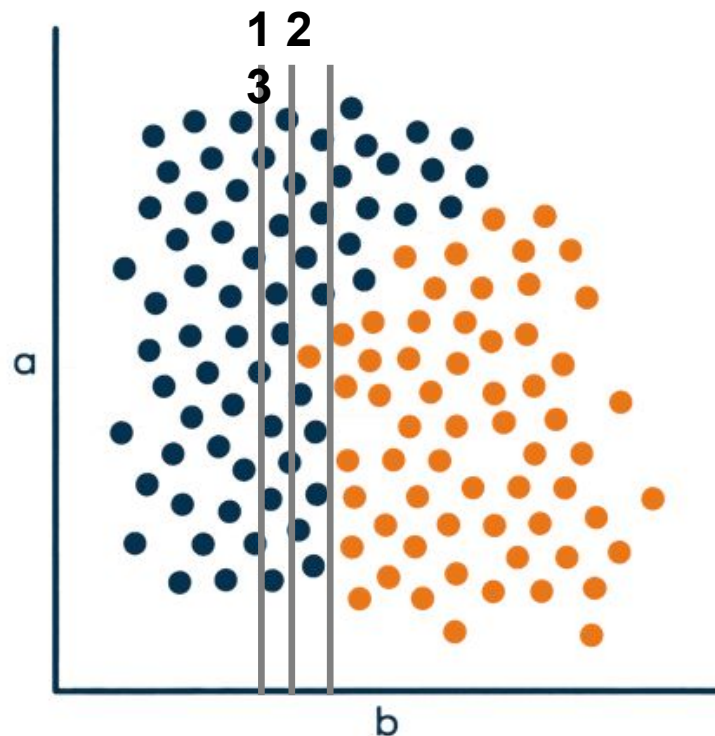
Mejoras de eficiencia en el algoritmo CART



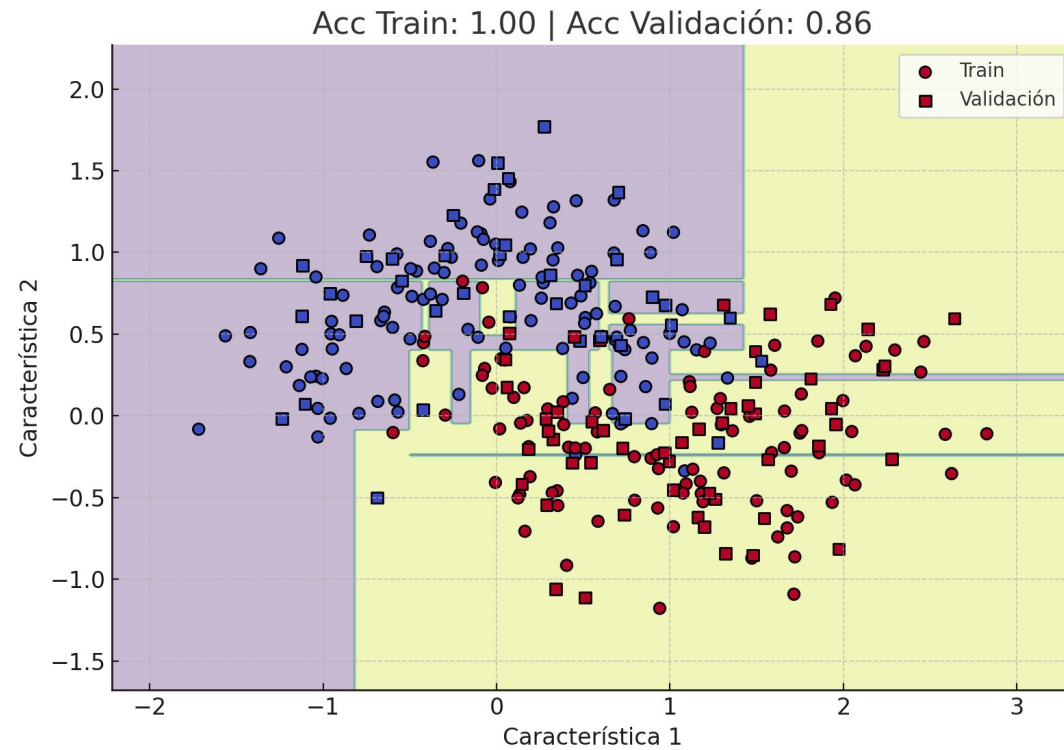
Mejoras de eficiencia en el algoritmo CART

Evaluar solamente umbrales en cambios de clase:

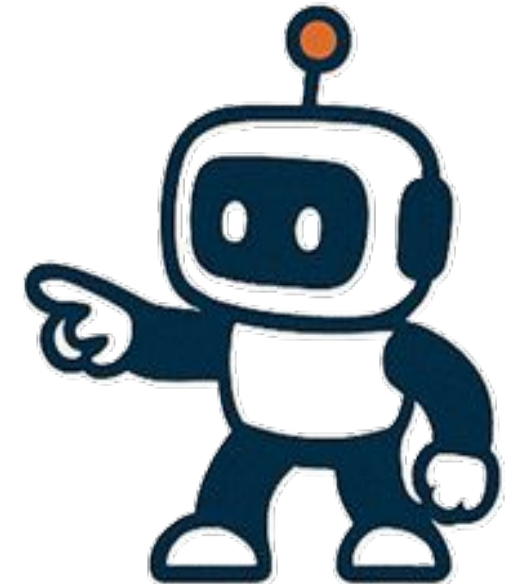
Ordenar los valores y solo evaluar umbrales en el cambio de clase



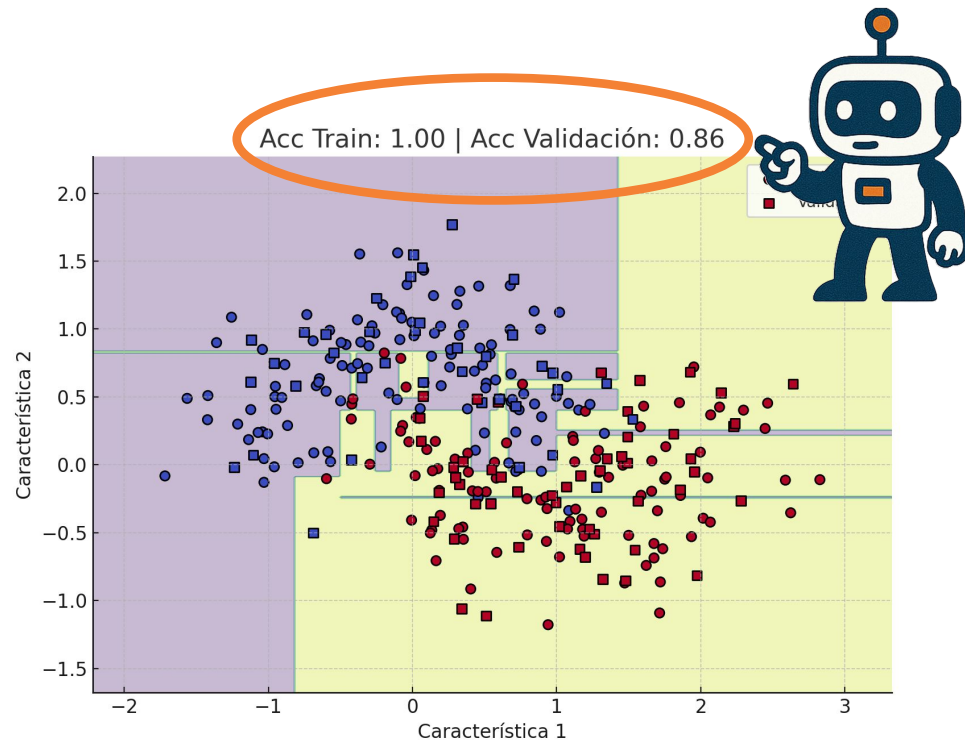
Ejemplo real árbol de decisión



¿Opiniones?



¿Como evaluar el rendimiento?



1) **Definir la métrica** y calcular sobre el conjunto de entrenamiento y validación.

2) **Validación cruzada** (cross-validation): Usarla para estimar rendimiento más estable. (K-fold)

3) **Gap de generalización**: diferencia entre rendimiento en entrenamiento y validación.

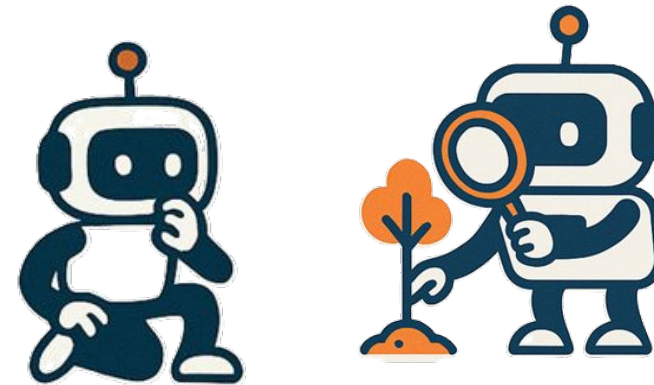
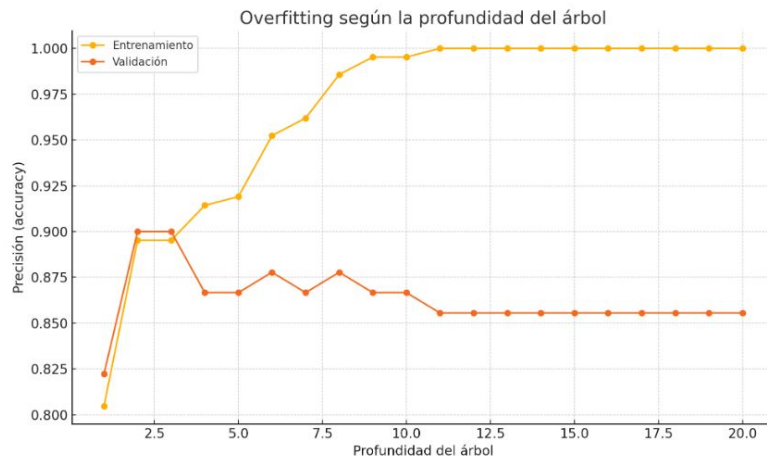
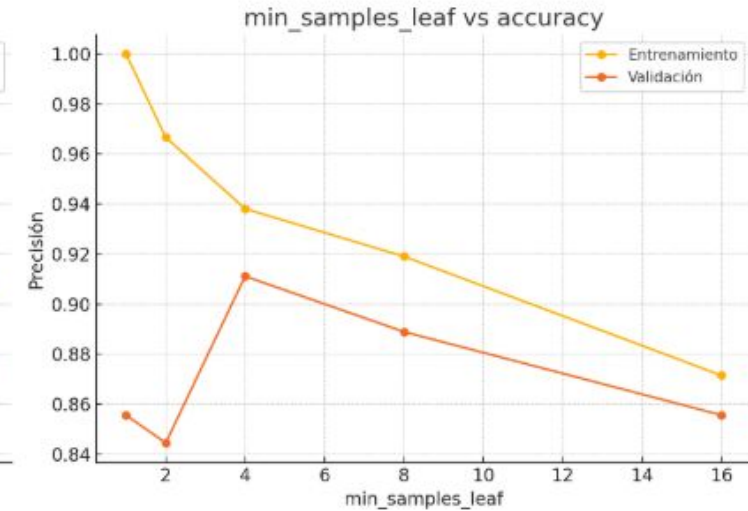
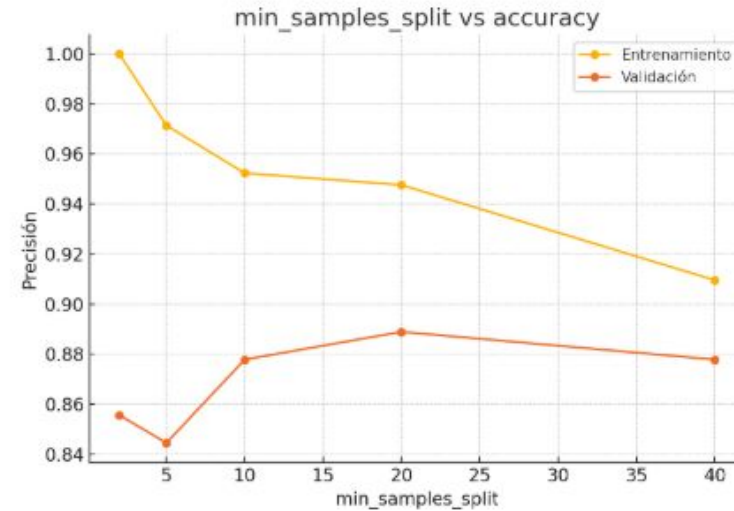
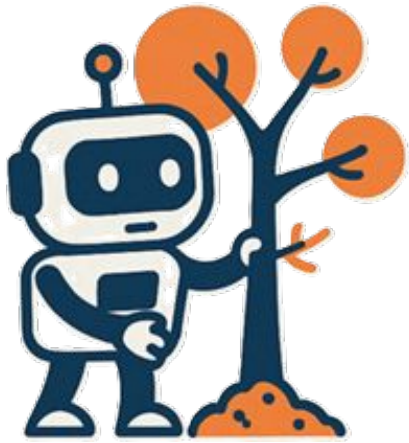
- **Gap grande** indica **sobreajuste**

- **Gap pequeño** pero **baja precisión** indica **subajuste**

4) **Curvas de validación**: Cómo varía el rendimiento al cambiar un hiperparámetro*.

*evaluar rendimiento vs complejidad (número de nodos, profundidad, min puntos por Split...).

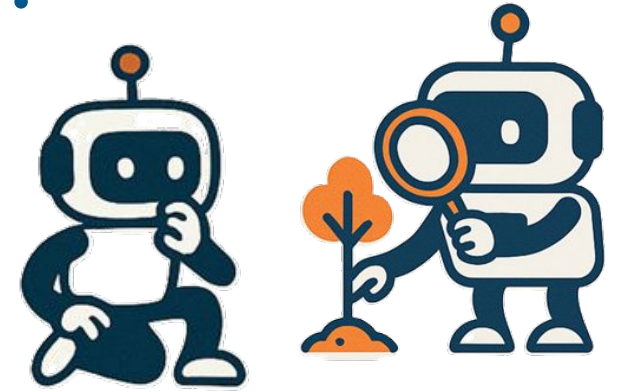
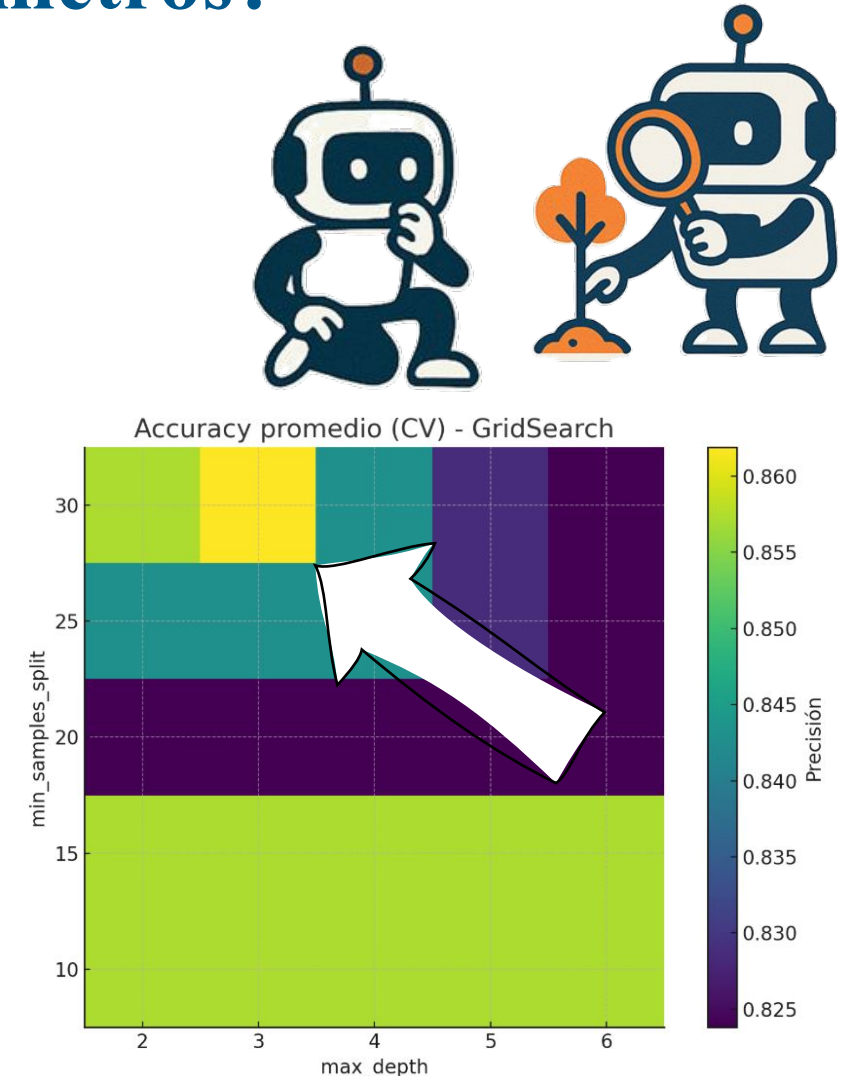
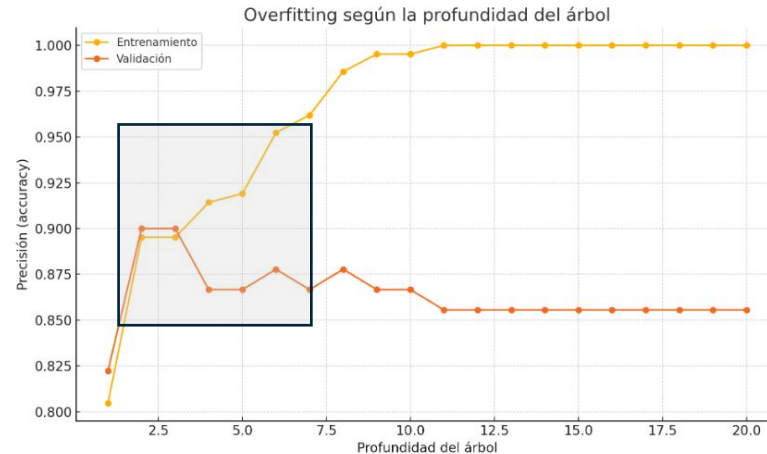
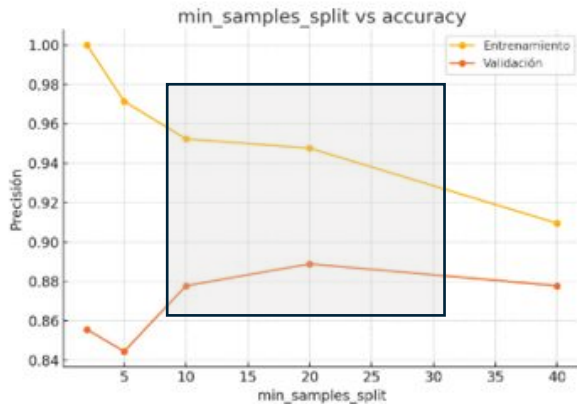
Ajuste de hiperparámetros



¿Y si queremos evaluar varios hiperparámetros?

Grid Search (búsqueda de rejilla):

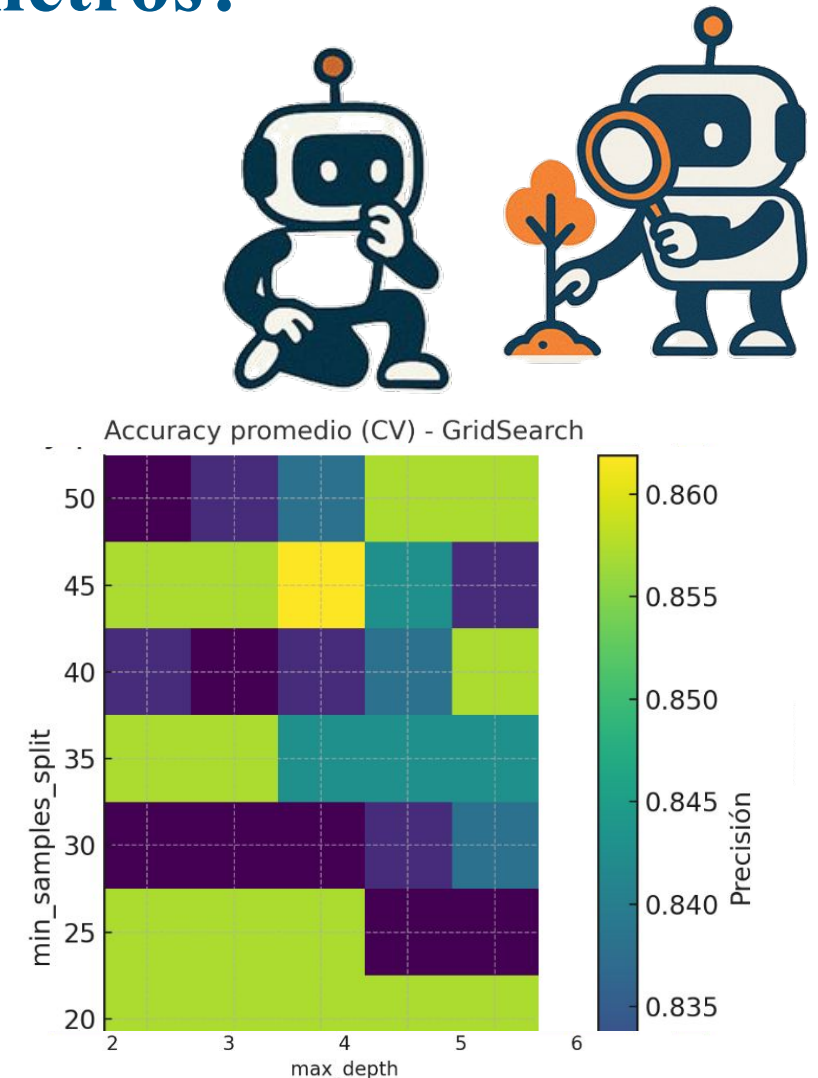
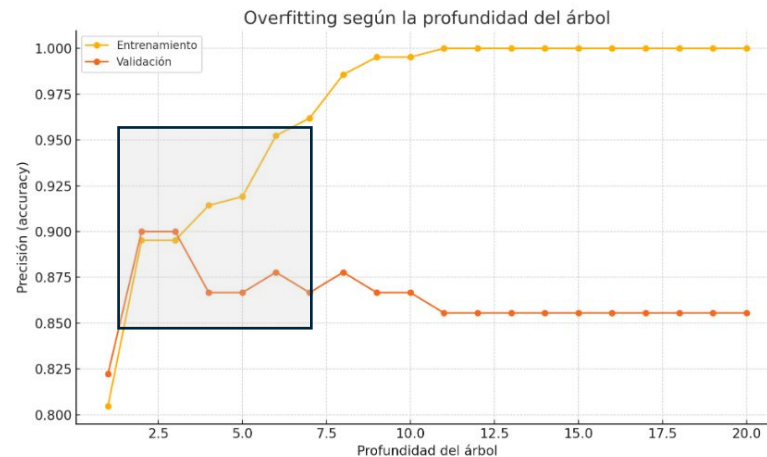
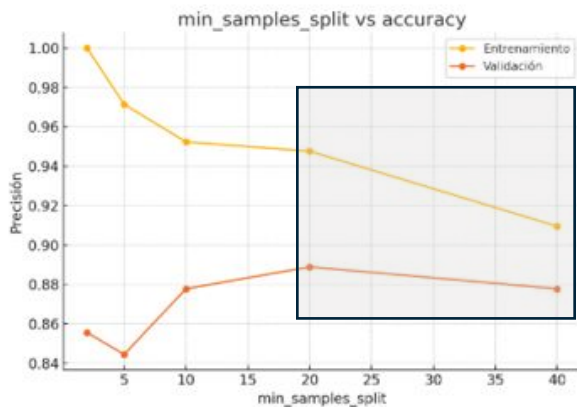
- 1) plotear un rango amplio del hiperparametro por separado (como en esta slide)
- 2) elegir un rango más acotado donde el hiperparámetro funciona bien, **pero darle margen!!** Ya que lo que muestra **overfitting** en el hiperparámetro individual podría ser compensado con los otros que pruebes!!!



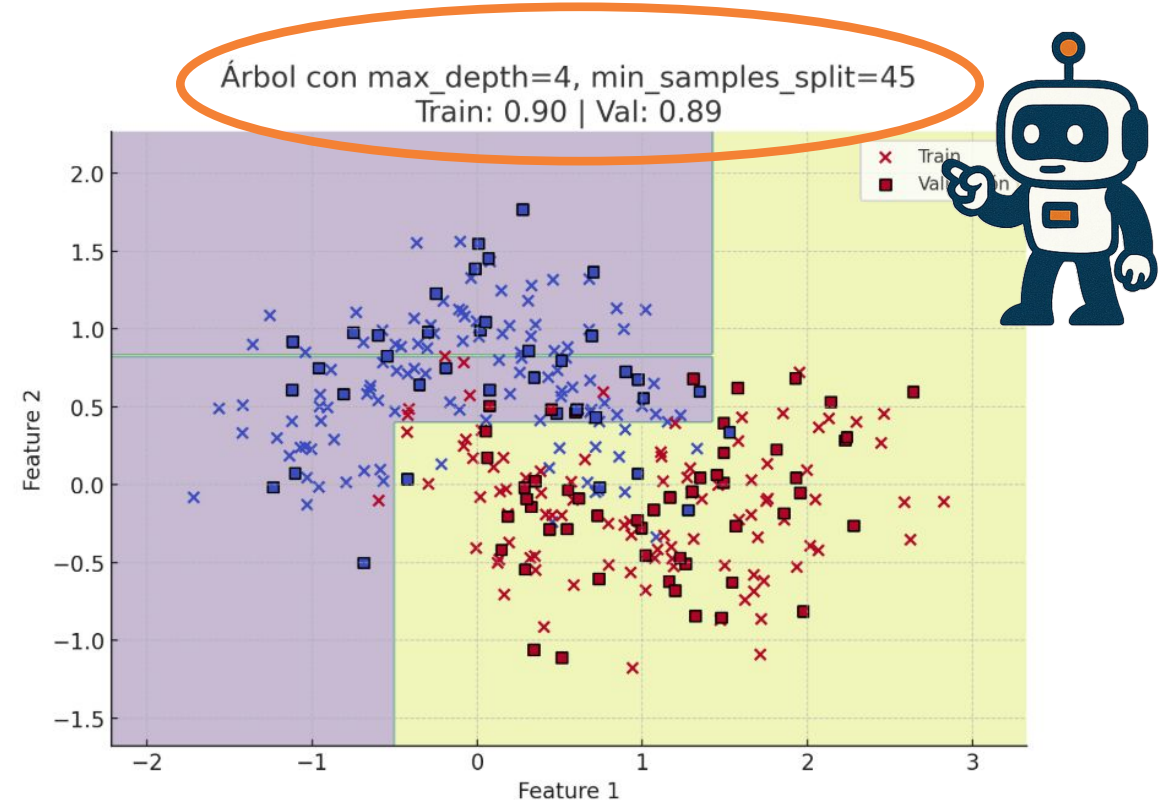
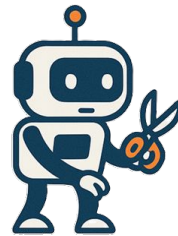
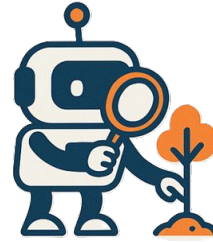
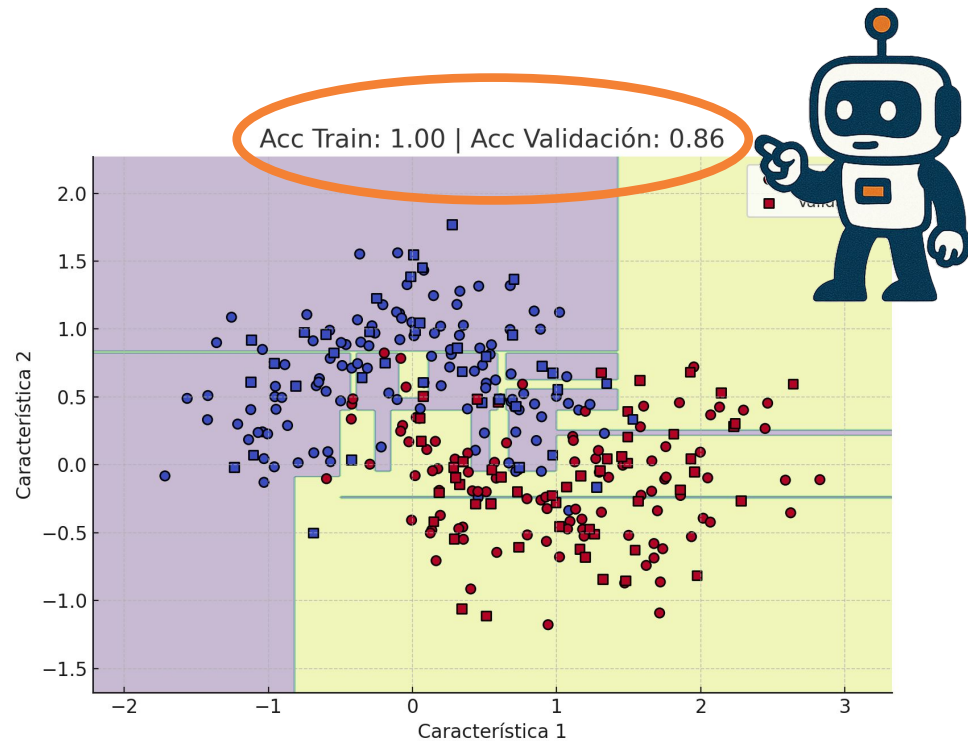
¿Y si queremos evaluar varios hiperparámetros?

Grid Search (búsqueda de rejilla):

- 1) plotear un rango amplio del hiperparametro por separado (como en esta slide)
- 2) elegir un rango mas acotado donde el hiperparametro funciona bien, **pero darle margen!!** Ya que lo que muestra **overfitting** en el **hipermarametro individual** podría ser compensado con los otros que pruebes!!!



Ejemplo



Resumen DT

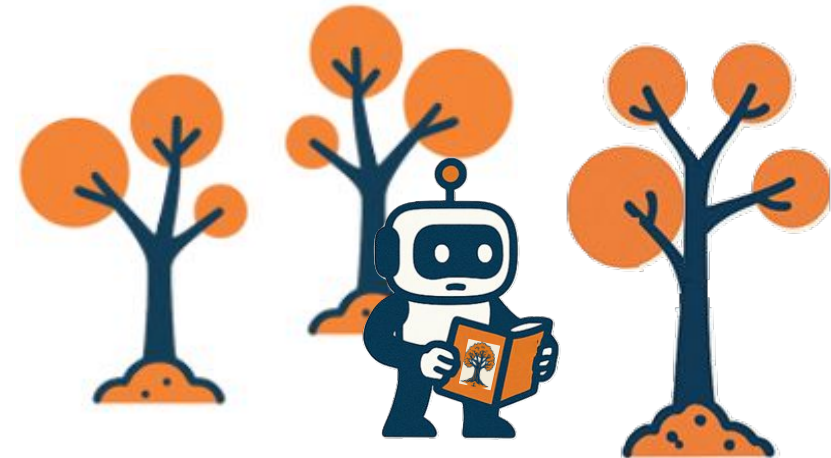
Un árbol de decisión es fácil de entender, pero también **propenso a sobreajustar**.

Aplicamos técnicas de **pre-poda** para limitar su complejidad, y reducir overfitting

Podemos optimizar varios de estos hiperparámetros para reducir overfitting sin bajar mucho la precisión.

....

¿Podemos seguir mejorando estos métodos?



Bootstrap

1) Definición:

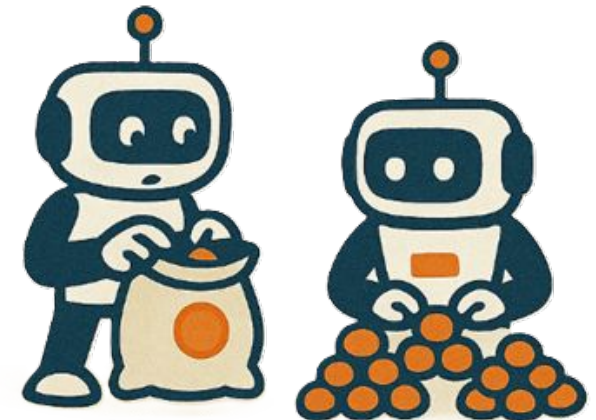
- Bootstrap es un método de muestreo con reemplazo.

2) Cómo funciona:

- A partir de un conjunto de datos de tamaño n , se generan nuevas muestras también de tamaño n , **seleccionando con reemplazo**.
- Algunos puntos aparecen varias veces, otros no aparecen.

3) Aplicación en Machine Learning:

- Permite entrenar múltiples modelos sobre datos distintos (aunque derivados del mismo dataset).
- Base del **bagging** y los **Random Forests**.



¿Porqué Bootstrap mejoraría DT?

- ¿Por qué usar bootstrap en árboles?
 - Genera **variedad entre árboles** → ayuda a reducir la **varianza del modelo**.
 - Hace que cada árbol "vea" una versión ligeramente distinta del problema.
 - Mejora la **generalización**, modelo más **estable** y menos sensible a outliers o ruido.
- Generalmente se extraen muestras es del mismo tamaño que el dataset original:
 - En promedio, cada muestra bootstrap contiene aproximadamente el **63%** de los datos únicos originales.
 - El **resto (~37%) se puede usar como conjunto de validación interna** ("out-of-bag").
- **En datasets muy grandes, se reduce el tamaño de estas muestras (y generalmente no se usa Bootstrap sino submuestreos aleatorios sin remplazo)**

Random Forests

- Conjunto de árboles de decisión entrenados con variaciones aleatorias
- Promediar las predicciones de arboles individualmente mas débiles, da una predicción mas robusta



¿Cómo funciona un Random Forest?

1) Se entrenan sobre subconjuntos aleatorios del dataset original

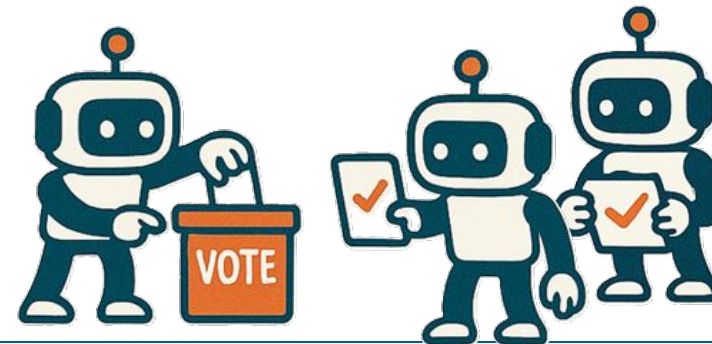
- Bootstrap: **Muestreo con reemplazo (repite valores!)** del conjunto de entrenamiento original.
- Cada árbol en Random Forest se entrena con una muestra del mismo tamaño que el dataset original.

2) ¡No se evalúan los umbrales en todas las características!

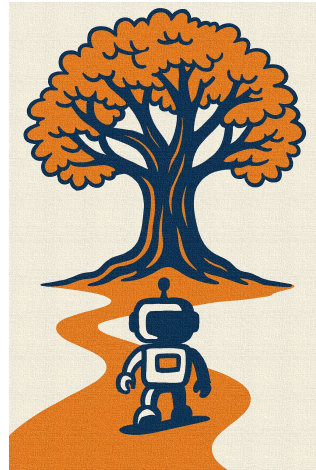
- En **cada** división de **nodo**, se consideran solo un **subconjunto aleatorio de variables**

3) Cada árbol **clasifica el dato por separado, y vota**. Al valor se le asigna la clase con mayor número de votos.

*si queremos la probabilidad de cada clase en lugar de la clase, calculamos el % de árboles que ha votado esa clase, y esa será la p de esa clase



DT vs RFs



Decision Trees



Random Forests

Ventajas

- Interpretabilidad

Desventajas

- Propenso al overfitting
- Muy sensible a datos

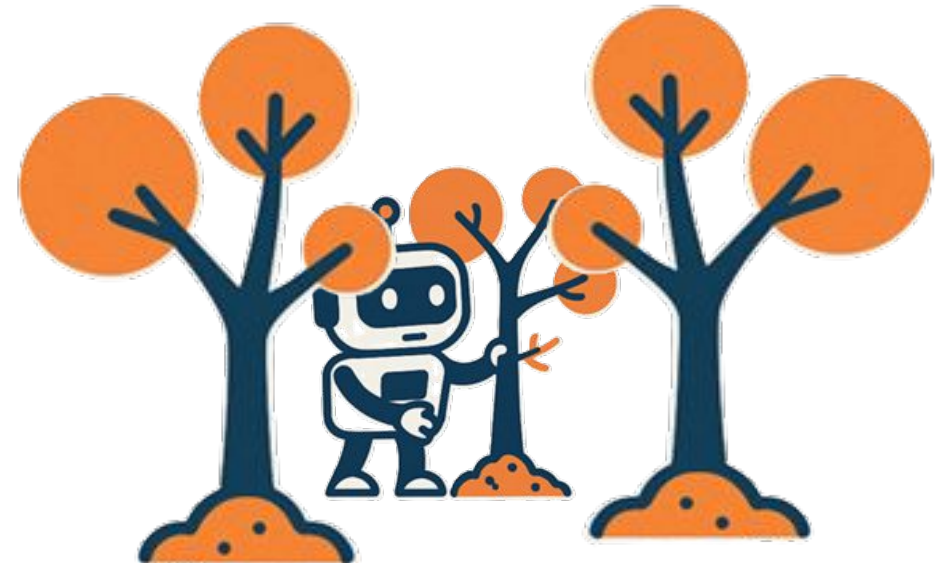
- Precisión y robustez
- Puede manejar grandes volúmenes de datos y variables
- Reduce overfitting
- Mayor coste computacional
- Menor interpretabilidad
- Posible overfitting en datasets pequeños

¿Hiperparámetros de un Random Forest?

Número de estimadores: número DTs a entrenar, más árboles, mejor estabilidad (hasta cierto punto)

max_features: clave para evitar árboles muy parecidos → más diversidad.
Cuanto menor, mas diversidad de DTs

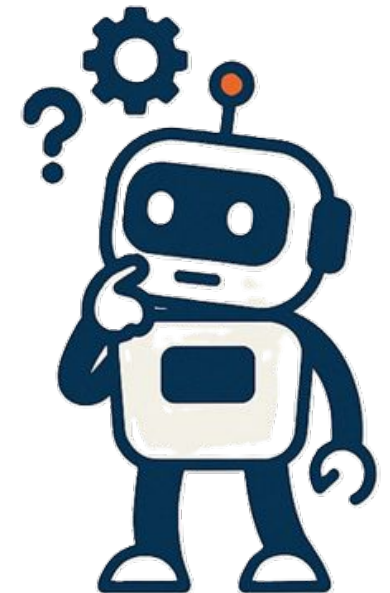
Ademas de todos los de los DTs: **max_depth**,
min_samples_split, **min_samples_leaf**.



¿Introducir aleatoriedad reduce overfitting?

- **Rompe la homogeneidad** entre árboles: cometen **errores distintos**.
- Al combinarlos, los **errores se atenúan** y **solo los patrones consistentes sobreviven** → mejor generalización.

- Caso extremo de **ExtraTrees**



Extra random Trees

- Caso extremo de random forests, con mayor aleatoriedad: Igual que RFs pero los umbrales a evaluar también son aleatorios
- Cada árbol es aún más débil individualmente
- Promediar muchos árboles débiles (si errores no correlados), los errores se diluyen y se mantiene lo que consistentemente ayuda a separar las clases



Comparación



Decision Tree



Random Forest



Extra Trees

Selección de
feature

Todas las features
disponibles

**Subconjunto aleatorio
de features**

**Subconjunto aleatorio de
features**

Selección de
umbral

Mejor umbral (óptimo)

Mejor umbral (óptimo)

**Evalúa un umbral aleatorio
por cada feature**

¿Qué optimiza?

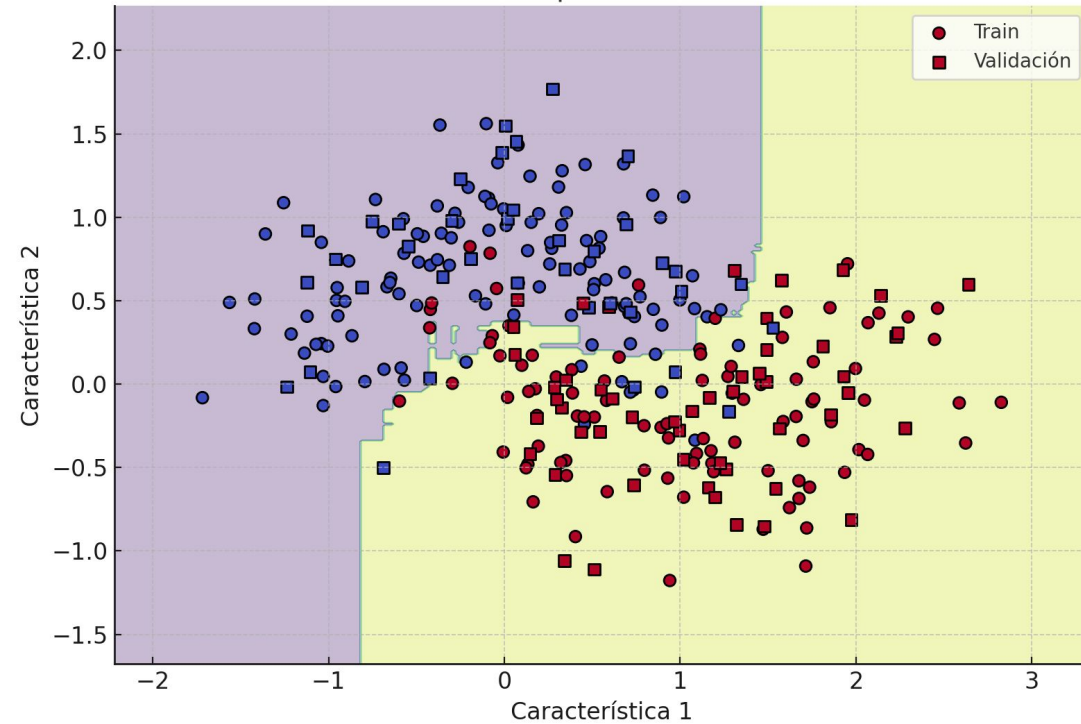
Mejor split global

Mejor split dentro de
features elegidas

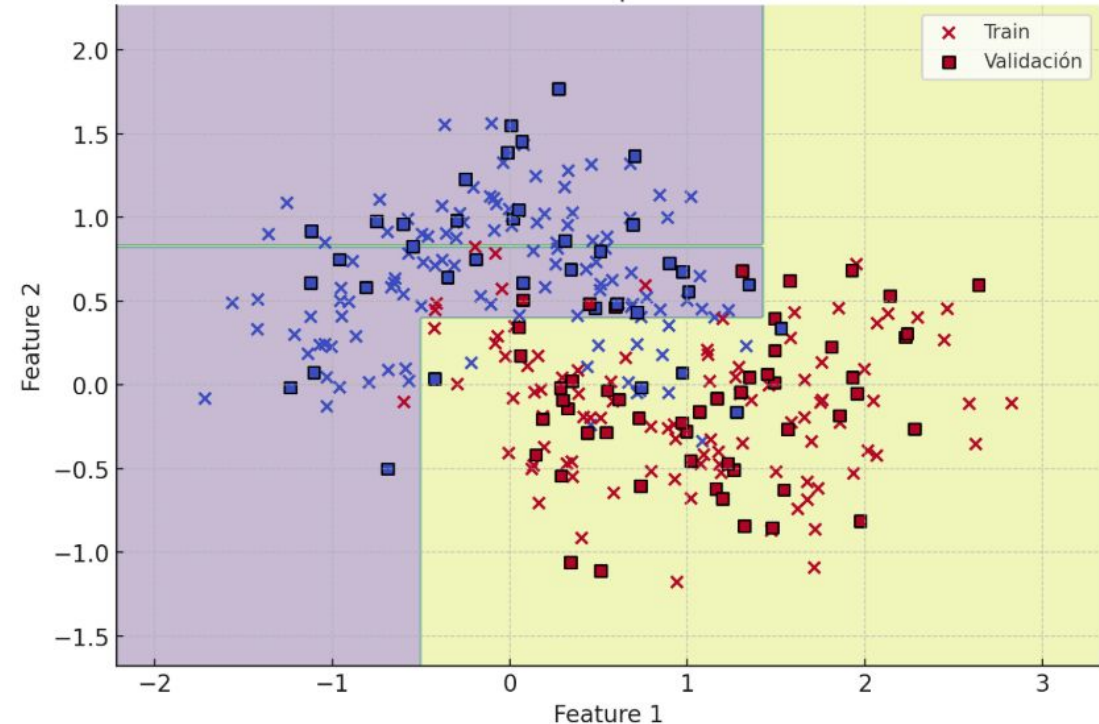
Mejor split entre esos
candidatos aleatorios

Ejemplo

Random Forest (sin rotar, max_depth=4)
Acc Train: 0.92 | Acc Validación: 0.89



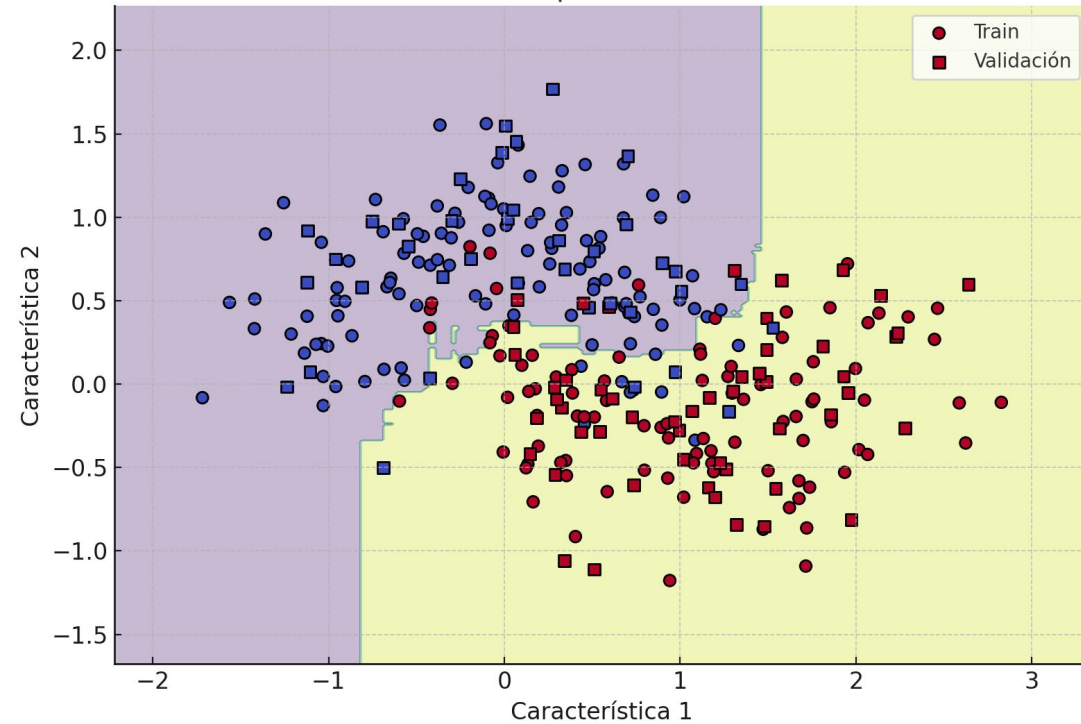
Árbol con max_depth=4, min_samples_split=45
Train: 0.90 | Val: 0.89



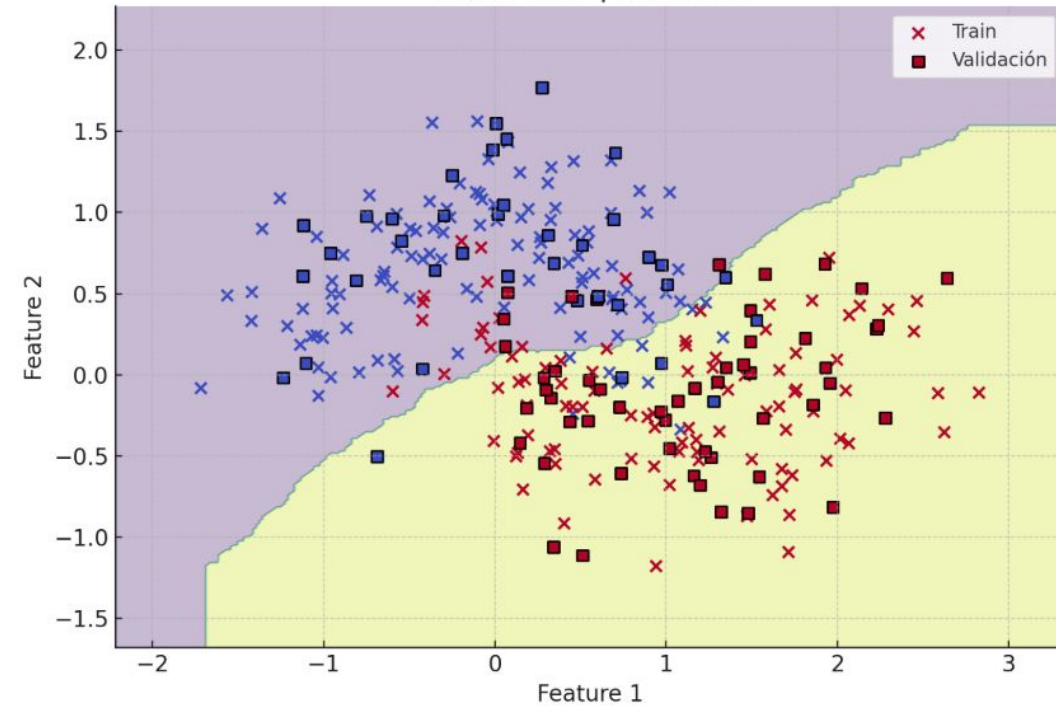
****Estamos usando solo dos características así que los random forests pierden un poco de potencial en este ejemplo**

Ejemplo

Random Forest (sin rotar, max_depth=4)
Acc Train: 0.92 | Acc Validación: 0.89

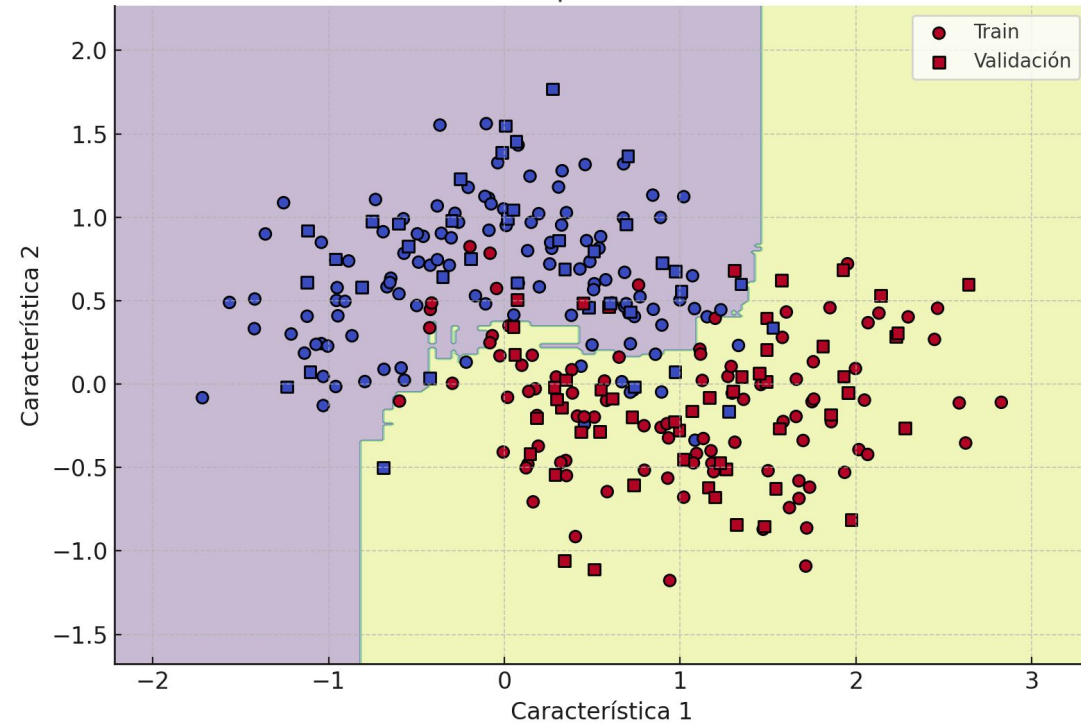


Extra Trees (500 árboles, max_depth=4)
Train: 0.87 | Val: 0.87

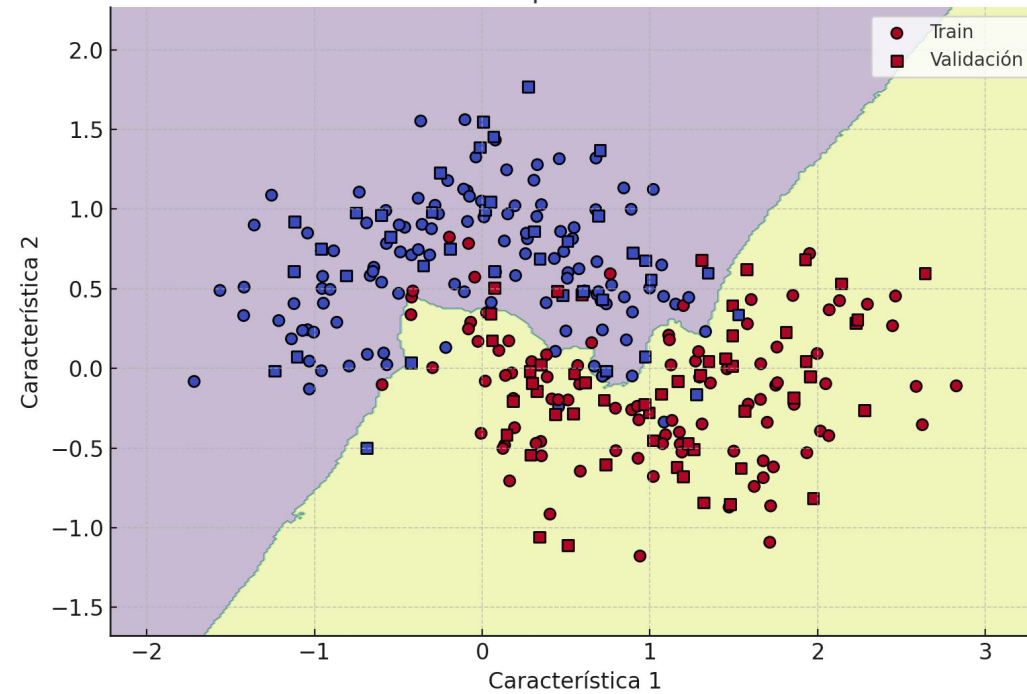


Ejemplo

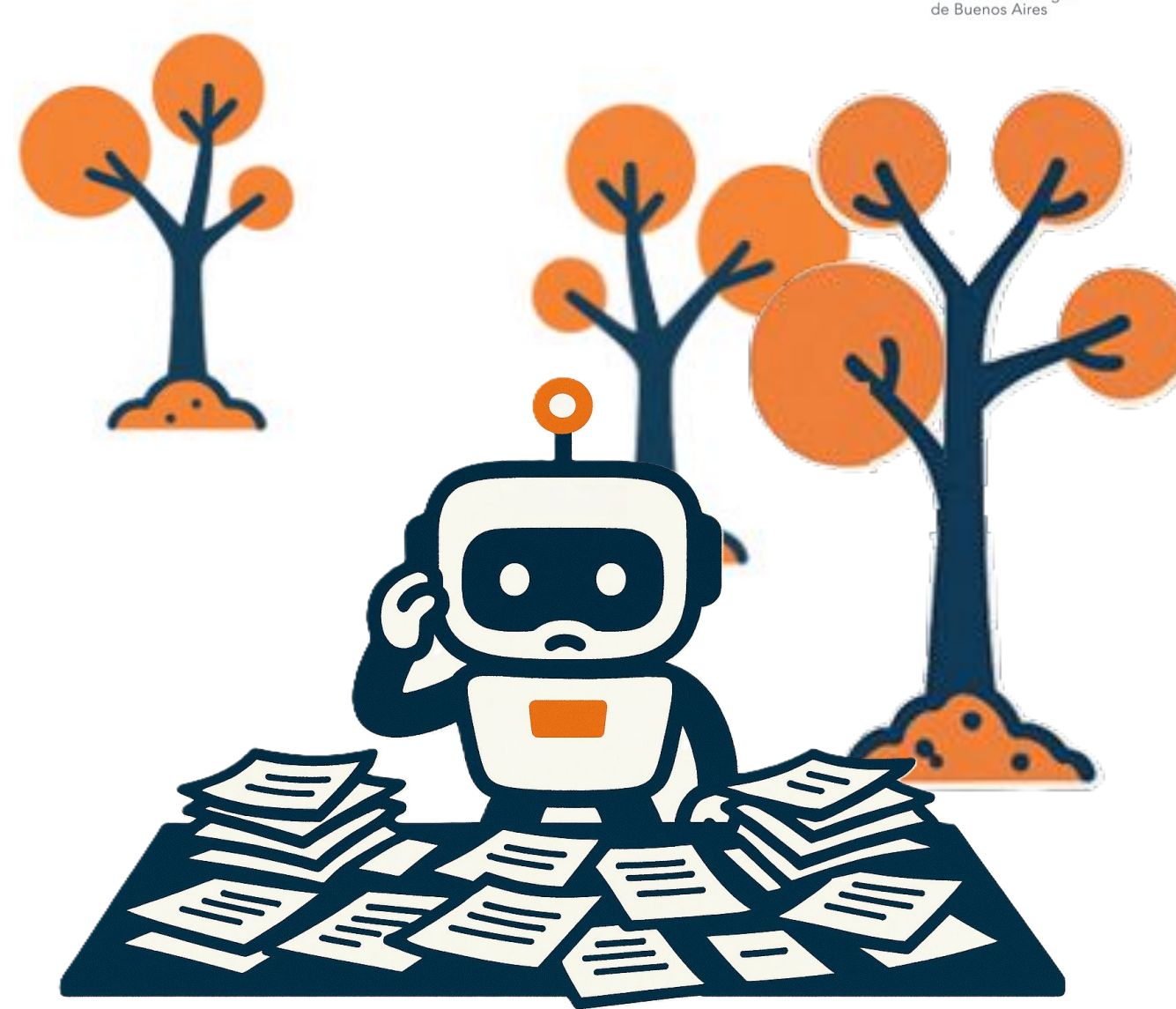
Random Forest (sin rotar, max_depth=4)
 Acc Train: 0.92 | Acc Validación: 0.89



Random Forest con Rotaciones Aleatorias
 Acc Train: 0.95 | Acc Validación: 0.91



¿Dudas finales?



¿Bagging aplicable a otros clasificadores?

- En la práctica, **los árboles de decisión son los más usados con Bagging**, porque son modelos de **alta varianza** (sensibles al dataset), y Bagging reduce justo eso.
- En cambio, modelos como regresión logística o SVM suelen tener **baja varianza y alto sesgo**, así que Bagging no aporta tanto.
- 👉 Por eso casi siempre se enseña Bagging con árboles: es donde realmente brilla (Random Forest es el ejemplo más famoso).
- **Bagging con árboles (muy útil).**
- **Bagging con modelos de baja varianza (menos útil).**

Stacking (stacked generalization)

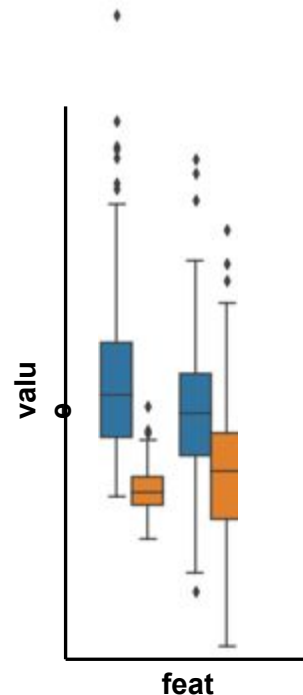
- Versión estándar: el meta-modelo recibe solo los outputs de los modelos base.
- Versión extendida (a veces llamada stacking with passthrough): el meta-modelo recibe tanto las predicciones de los modelos base como las features originales.
- → Esto permite que el meta-modelo “reconozca” en qué regiones del espacio de entrada cada modelo funciona mejor.

Importancia de las features

- Los DTs (y RFs) pueden también informarnos sobre qué variables son mas importantes (viendo cuales son las que mas utiliza)
- Calcular cuanta impureza reducen los nodos de cada feature:
 - cada vez que se seleccione un umbral, sumar la reducción de impureza que ese umbral produce y sumarlo a la reducción de esa variable.
 - Al final tienes un vector de reducciones de impureza que te informan de que variables son las mas informativa

Mejoras útiles para estos métodos

- **Selección de características (Feature Selection):** Reducir el número de variables de entrada puede eliminar ruido y reducir el sobreajuste.
 - Importancia de features del propio RF (feature_importances_)
 - Selección por varianza, correlación, discriminabilidad (graficar las características para cada clase para ver como de distintas son sus distribuciones -> que tan discriminante es esa variable)
 - Comprobar que variables están correladas entre ellas (correlation matrix) y quitar las que estén correladas entre ellas (todas menos una...)
- **Blending con RF y KNN...** interesante combinación ya que RF captura diferencias más globales mientras que KNN más locales. Potencialmente cometerán distintos errores por lo que combinar sus predicciones puede ser interesante
 - Entrenar RF y KNN por separado sobre los datos de train, generando una probabilidad cada uno.
 - Combinar sus predicciones para generar una nueva: por ejemplo con el promedio. O promedio ponderado. O métodos mas complejos



¿Gini o entropía para métodos avanzados?

- Ninguna de las dos: Log-Loss

$$\text{log-loss} = -[y \cdot \log(\hat{p}) + (1 - y) \cdot \log(1 - \hat{p})]$$

- Ventajas:

- Es derivable y por tanto permite cálculos de gradiente
- Crece rápidamente cuando la predicción es muy mala (por ejemplo, p estimada de 1 es 0 cuando la clase real es 1). Es **muy sensible al error con alta confianza**.

